

Introduction to Big Data

IN411 – M1S2 – 3 Credits

Course Contents

- Chapter 1: Overview of Big Data
- **Chapter 2: Hadoop Ecosystem**
- Chapter 3: Apache Spark Basics
- Chapter 4: Introduction to NoSQL Databases
- Chapter 5: Big Data Storage & Data Lake Concepts

Chapter 2:

Hadoop Ecosystem

Content

- Introduction to the Hadoop platform (HDFS, YARN, MapReduce)
- Hadoop distributed file system (architecture, reading/writing data)
- MapReduce paradigm & programming exercises
- Hadoop ecosystem tools introduction (Hive, Pig, HBase): short overview

Hadoop Platform Introduction

Introduction

- **Small** datasets are datasets that fit comfortably in RAM, leaving memory to spare for manipulation and transformations.
- They are usually no more than 2–4 GB in size, and complex operations like sorting and aggregating can be done without paging.
- Such tools have upper limits to their feasibility when working with datasets beyond a certain size.



Introduction

- **Medium** datasets are datasets that cannot be held entirely in RAM but can fit comfortably in a single compute's persistent storage.
- **Large** datasets are datasets that can neither fit in RAM nor fit in a single computer's persistent storage.

Introduction

- Two possible (and easy solutions) to not run into memory problems when processing data, are to:
- **Optimize Storage**
 - Store data in efficient memory formats (compression).
 - Store data in a database to allow for sql queries.
 - Use chunking to trim and thin out your data so it does fit in memory.
- **Scale** your problem:
 - Replace your existing equipment with faster, more efficient equipment (scale-up).
 - Hire more workers to work in parallel (scale-out).

Introduction

- The exponential growth of data (Big Data) exceeded the capabilities of traditional relational databases and centralized processing systems.
- Challenges included:
 - High cost of enterprise storage systems
 - Storage of unstructured and semi-structured data
 - Processing of unstructured and semi-structured data efficiently
 - Scalability limits
 - Fault tolerance limits
 - Throughput limits

Introduction

- **Apache Hadoop** emerged as an open-source framework enabling:
 - A platform for big data
 - Running applications on large clusters of commodity hardware
 - Distributed storage
 - Distributed processing
 - Fault tolerance

Core Design Principles

- Hadoop is based on the following principles:
 - **Data locality:** Move computation to where data resides, because it is more efficient than moving large data.
 - **Horizontal scalability:** Scale by adding nodes.
 - **Fault tolerance:** Expect hardware failures.
 - **Batch-oriented processing:** jobs are collected, placed in a queue for processing.
 - **Write-once, read-many** data access pattern.
 - **Resource sharing:** which is the motivation behind distributed Systems.
 - **File System:** which is responsible for the organization, storage, retrieval, naming, sharing, and protection of files.

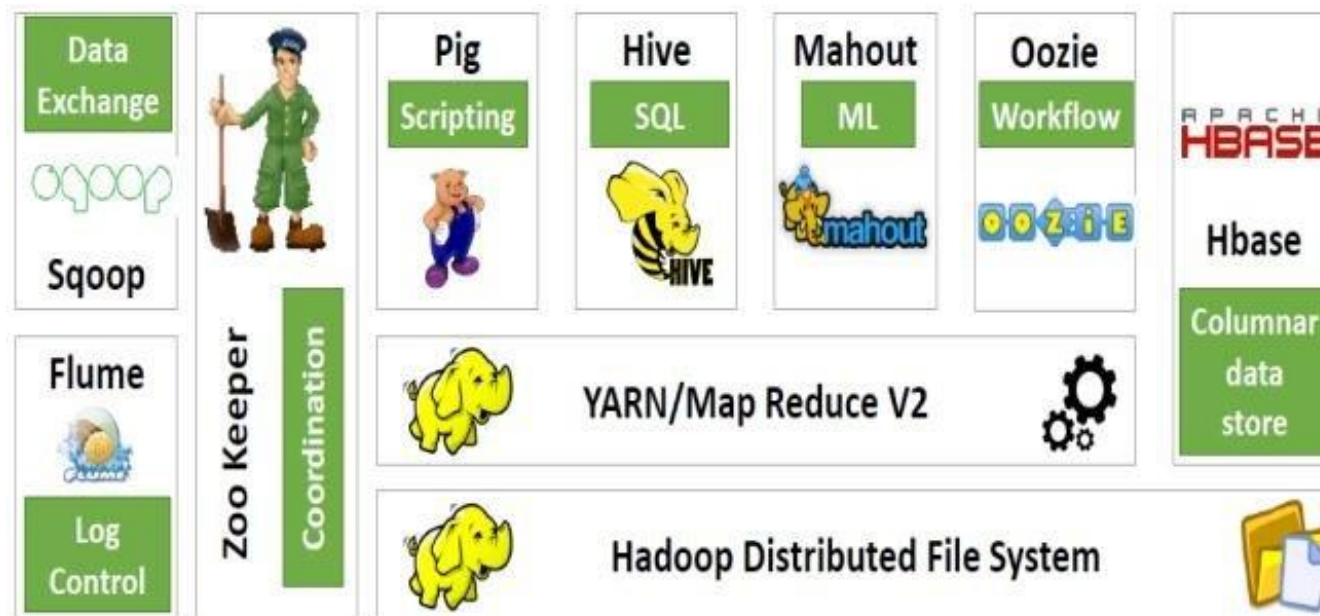
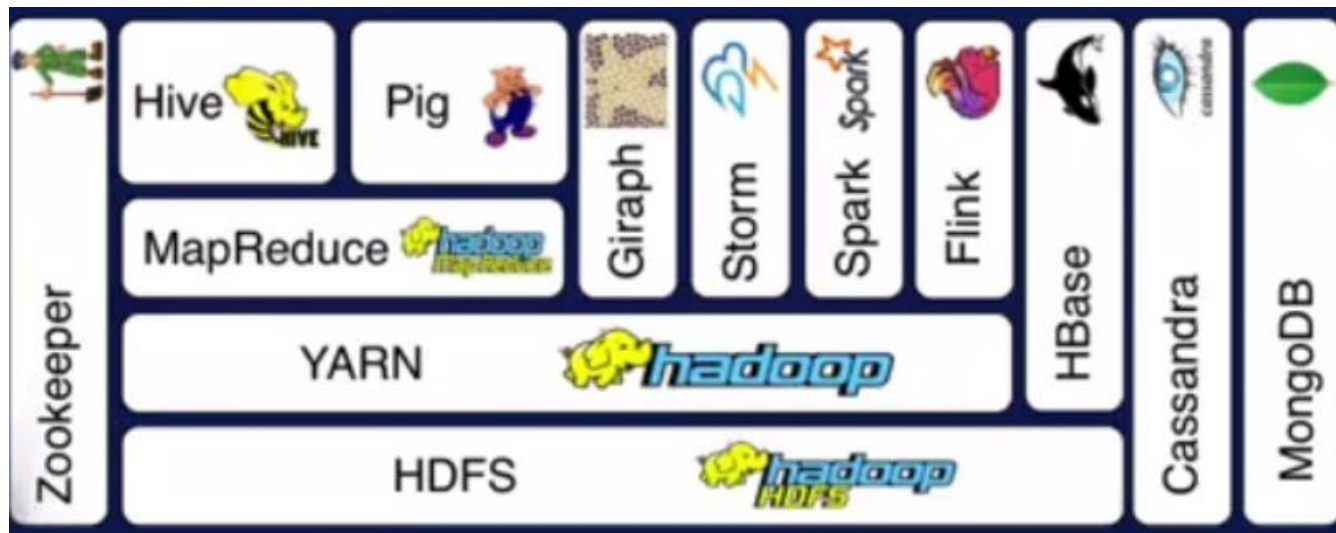
Hadoop Logo



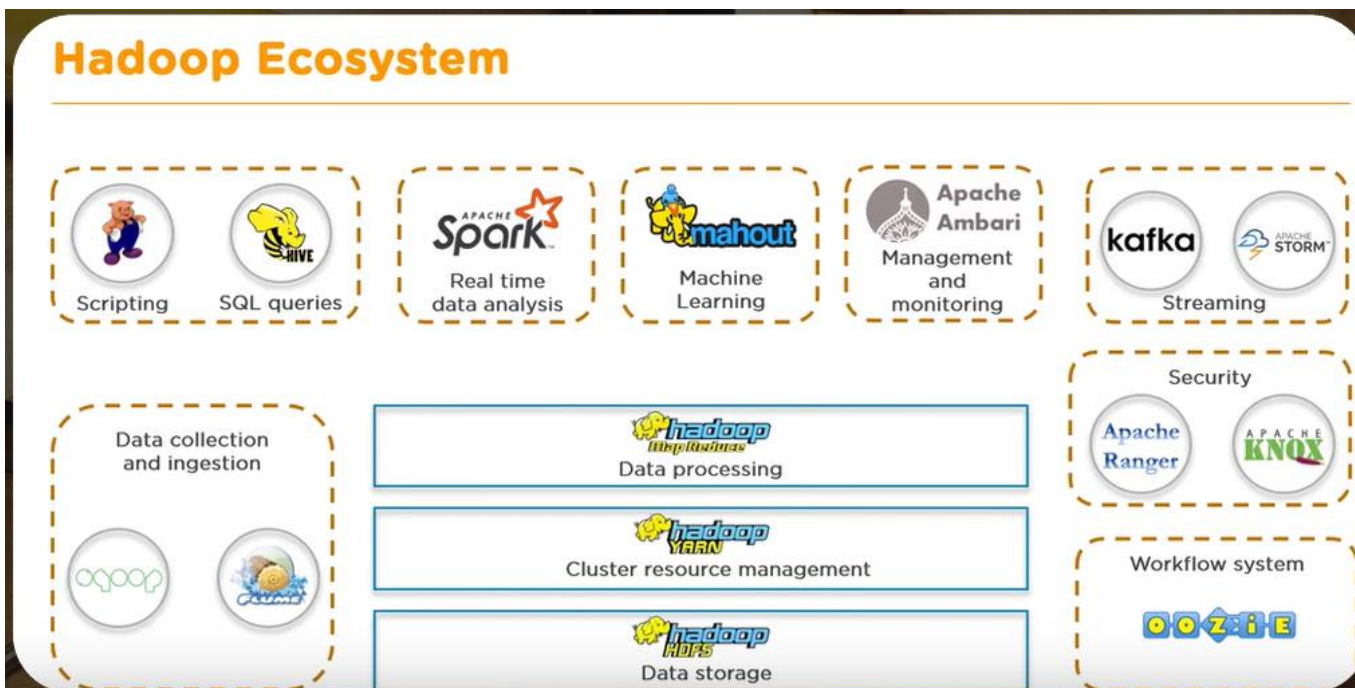
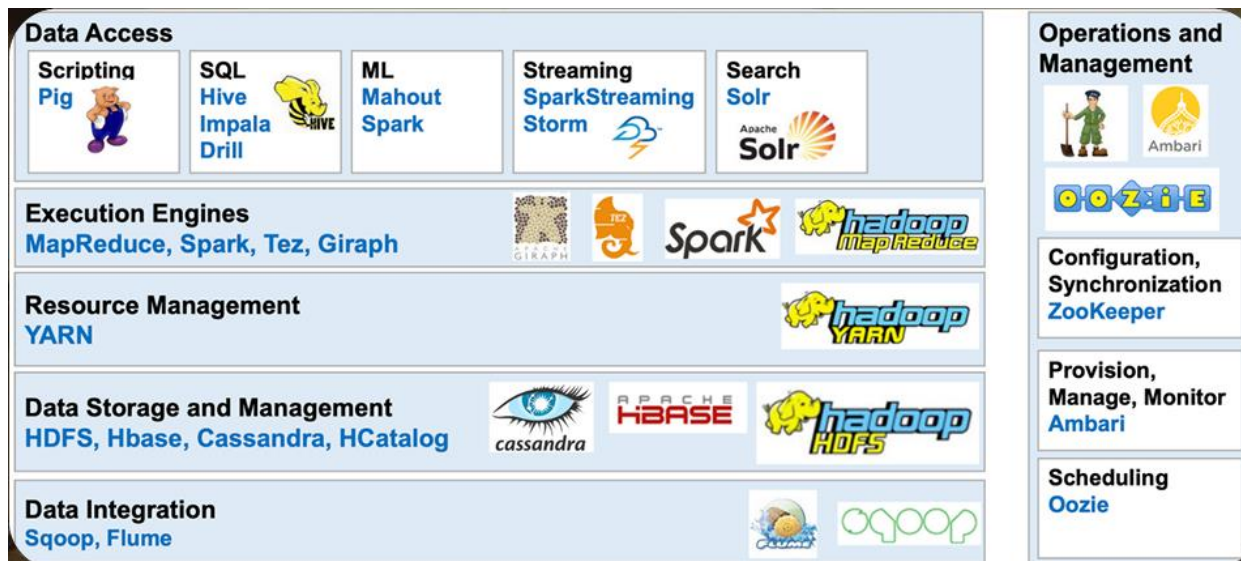
Hadoop Architecture Overview

- Apache Hadoop includes :
 - **HDFS**: a distributed filesystem for Big Data **storage**.
 - **YARN**: for Resource Management.
 - **Map/Reduce**: a computing engine implementing programming model for Big Data **processing**.
 - **Data Access and Analytics Tools**: Spark, Pig, Mahout, Tez, Giraph, Flink, DataMeer, Alpine Miner, ...
 - **NoSQL and Storage Extensions**: Hive, Hbase, Cassandra, MongoDB, Impala, drill, Hcatalog, ...
 - **Data Ingestion and Integration Tools**: Sqoop, Flume, Kafka, Storm, Solr...
 - **Workflow, Coordination, and Management**: Oozie, ZooKeeper, Ambari...
 - **Hue**: The Hadoop desktop GUI, allows launching Data Browsers and Query Tools.
 - **Security**: Ranger, Knox, ...

Hadoop Architecture Overview



Hadoop Architecture Overview



Advantages

- Supports use of inexpensive, commodity hardware
- No RAID needed. Also, the servers need not be the latest and greatest hardware.
- Provides for simple, massive parallelism
- Provides resilience by replicating data and eliminating tape backups
- Provides locality of execution, as it knows where the data is
- Software free
- High quality support available at modest cost
- High quality training available at modest cost
- Certification available
- Easy to support when using GUI such as Cloudera Manager or Ambari
- Add-on tools available at relatively low cost, or in some cases no cost
- Evolving technology with a high degree of interest around the world

Use cases

- Yahoo and Facebook are two well known companies that use Hadoop.
- Yahoo has over 42,000 nodes.
- Largest amount of disk storage in use: Facebook over 100 Petabytes, Growing at over 1/2 Petabyte per day.

Security in Hadoop Ecosystem

■ Authentication

- Kerberos-based authentication
- Secure cluster access

■ Authorization

- Apache Ranger / Apache Sentry
- Fine-grained access control

■ Data Protection

- HDFS encryption at rest
- TLS for data in transit
- Audit logging

Hadoop Deployment Models

- **On-Premises Clusters**
- **Cloud-based Hadoop** (Amazon EMR, Azure HDInsight)
- **Hybrid architectures**

Hadoop distributed file system (HDFS)

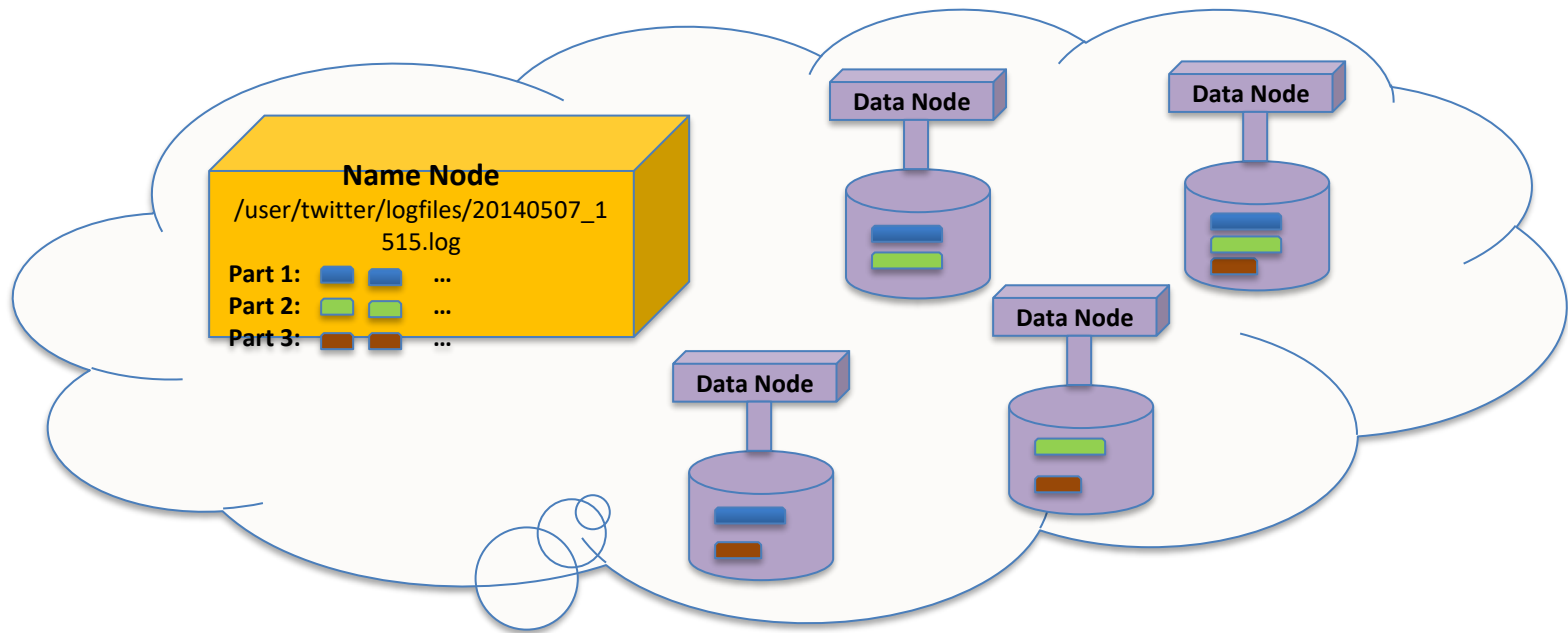
Introduction

- The **Hadoop Distributed File System (HDFS)** is a scalable, fault-tolerant distributed storage system designed to store **very large datasets** (terabytes to petabytes) across clusters of commodity hardware.
- HDFS is optimized for **high-throughput batch access** rather than low-latency data access.
- HDFS is a core component of the Hadoop ecosystem and provides the storage foundation for higher-level processing frameworks such as **MapReduce, Spark, Hive, and HBase**.



HDFS

- The key concept is to split the data and store it across collection of machines known as a cluster.
- Then the data is processed where it is actually stored.

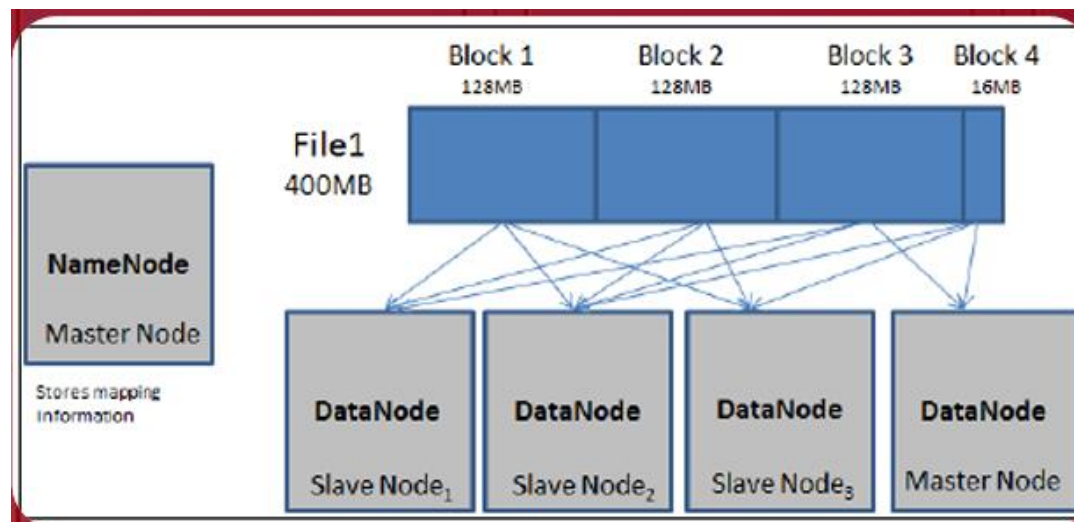


HDFS

- To store data, Hadoop utilizes its own distributed filesystem, HDFS, which makes data available to multiple computing **nodes**.
- A distributed file system is a file system that can store large files spread across the nodes of a cluster.
- HDFS works by breaking large files into smaller pieces called **blocks**.
- When data is stored in Hadoop, the **NameNode** file automatically stores and replicates the data (3 times) in multiple blocks (64 MB or 128 MB by default) across the various **DataNode**.
- This is done to ensure fault tolerance and high availability.

HDFS

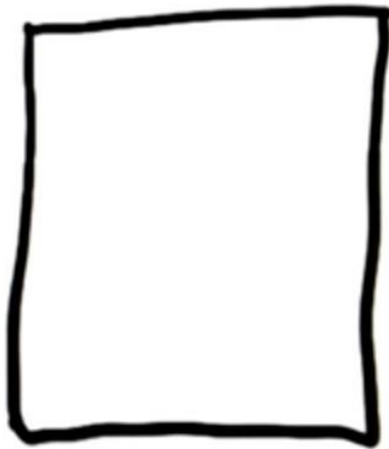
- The master node (aka **NameNode**) stores all the metadata, access rights, mapping and location of files and blocks, and so on.
- The slaves (aka **DataNode**) are nodes where the actual data is stored.
- All the requests go to the master and then are handled by the appropriate slave node.



HDFS

HDFS

mydata.txt



150MB

HDFS

mydata.txt



150MB

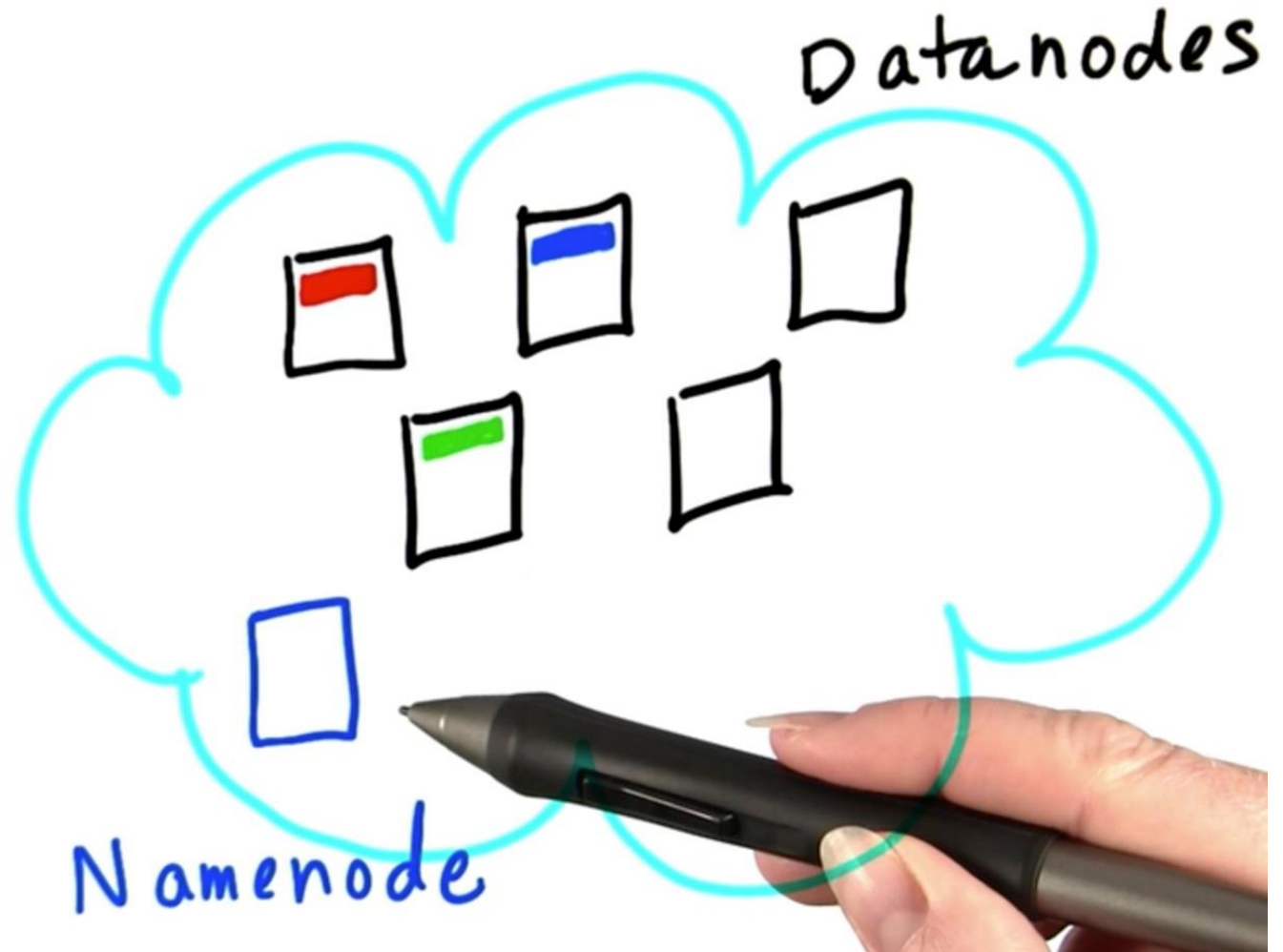
HDFS

HDFS



HDFS

HDFS



HDFS

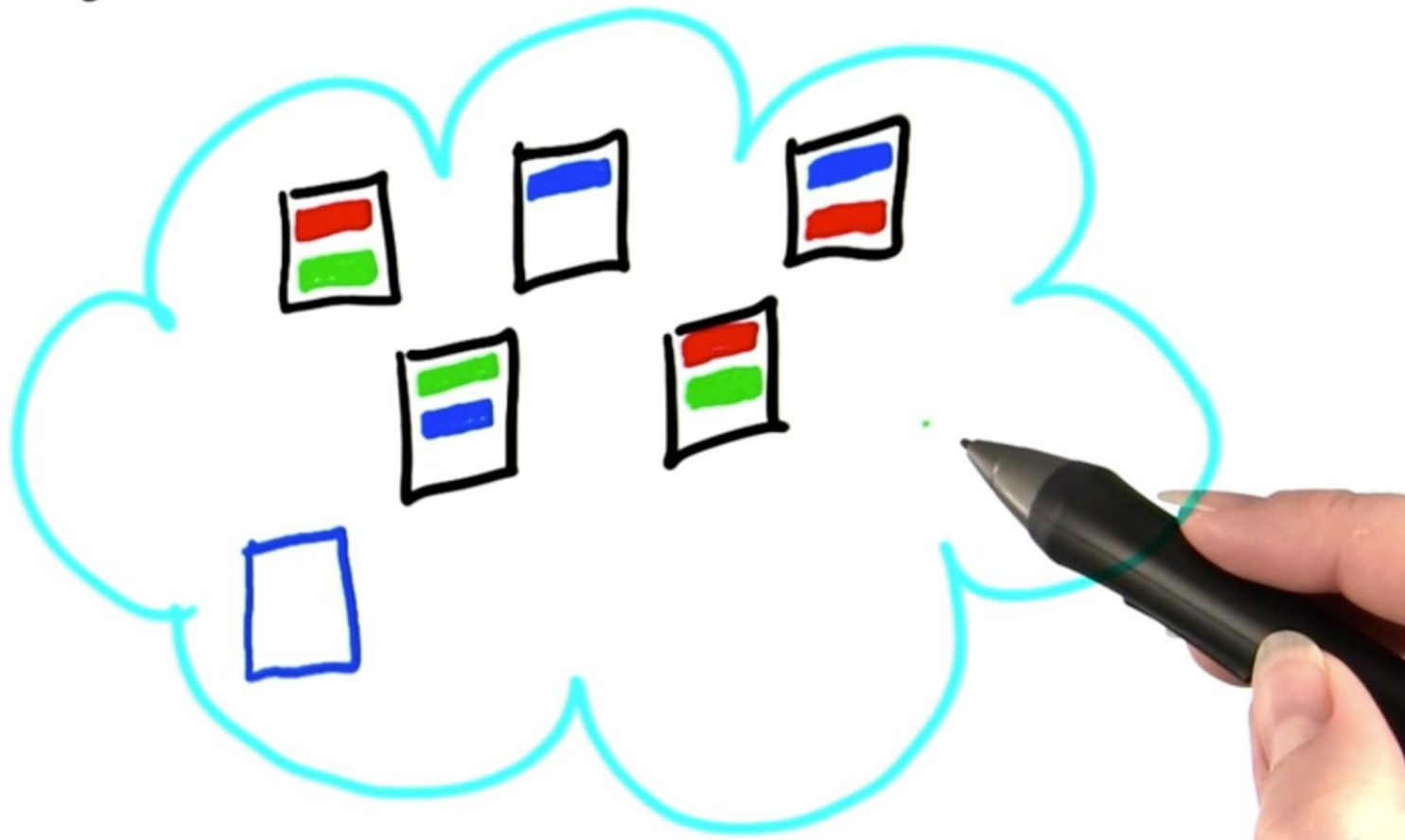
IS THERE A PROBLEM?

- ☒ NETWORK FAILURE
- ☒ DISK FAILURE ON DN
- ☐ NOT ALL DN USED
- ☐ BLOCK SIZES DIFFER
- ☒ DISK FAILURE ON NN



HDFS

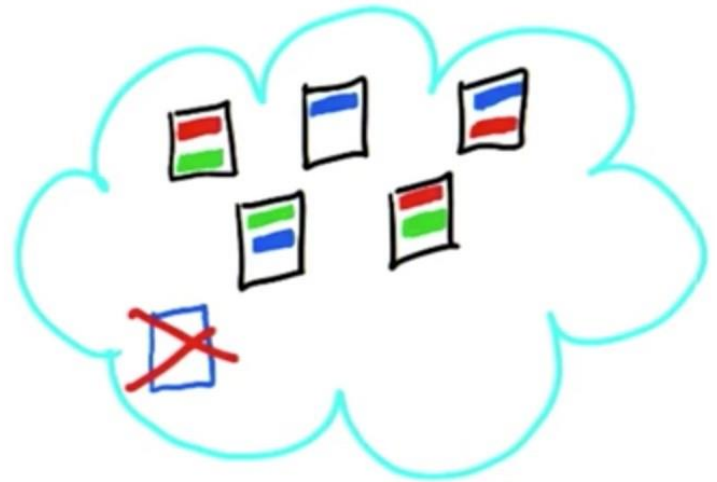
DATA REDUNDANCY



HDFS

ANY PROBLEMS NOW?

- ☒ DATA INACCESSIBLE
- ☒ DATA LOST FOREVER
- ☐ NO PROBLEM



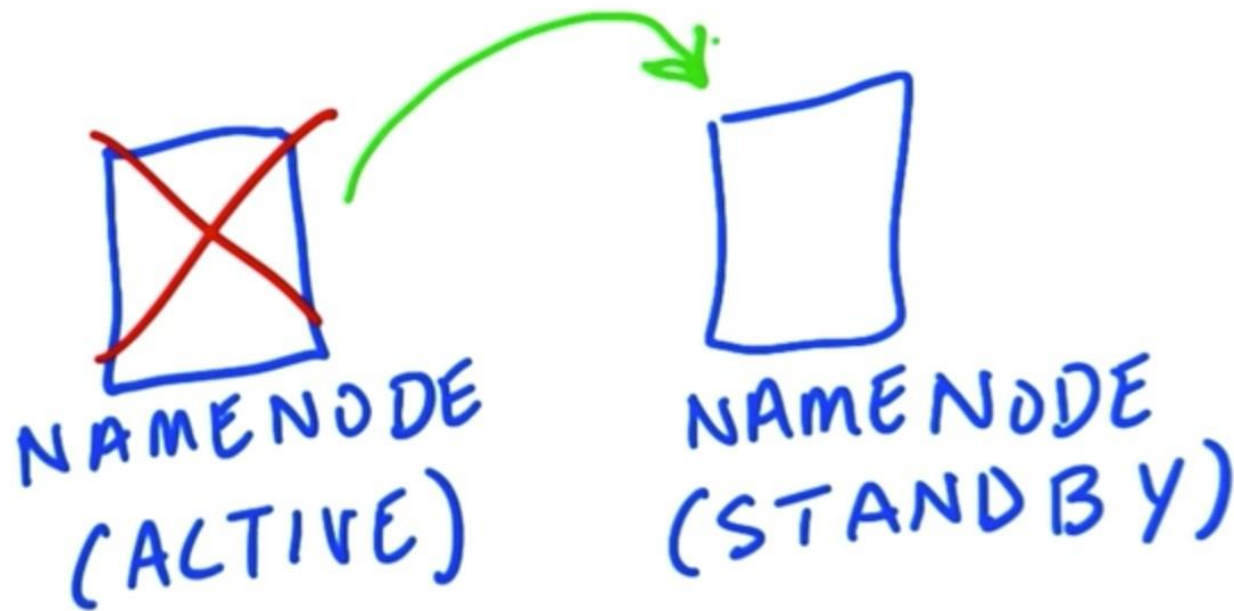
HDFS

NAME NODE HIGH AVAILABILITY

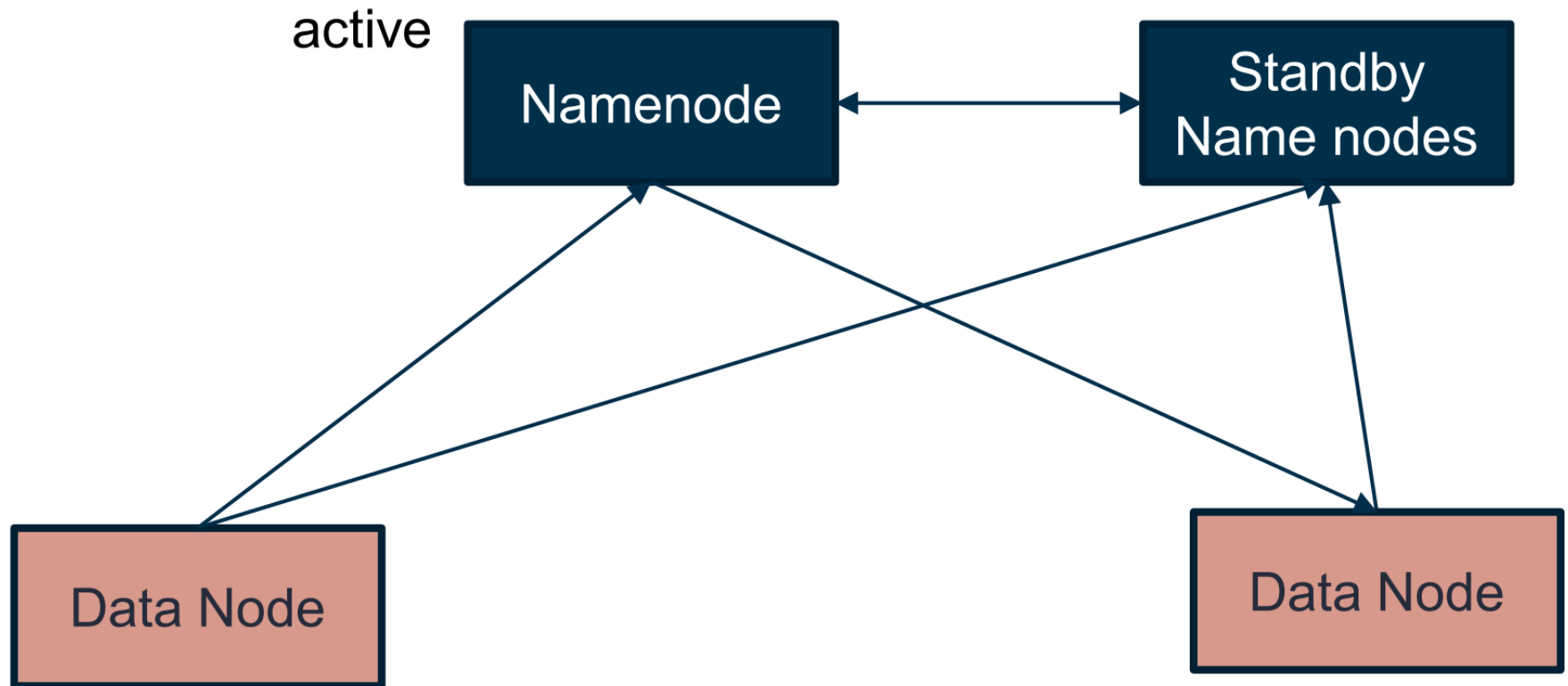


HDFS

ACTIVE AND STANDBY
NAMENODE



Keeping a standby namenode



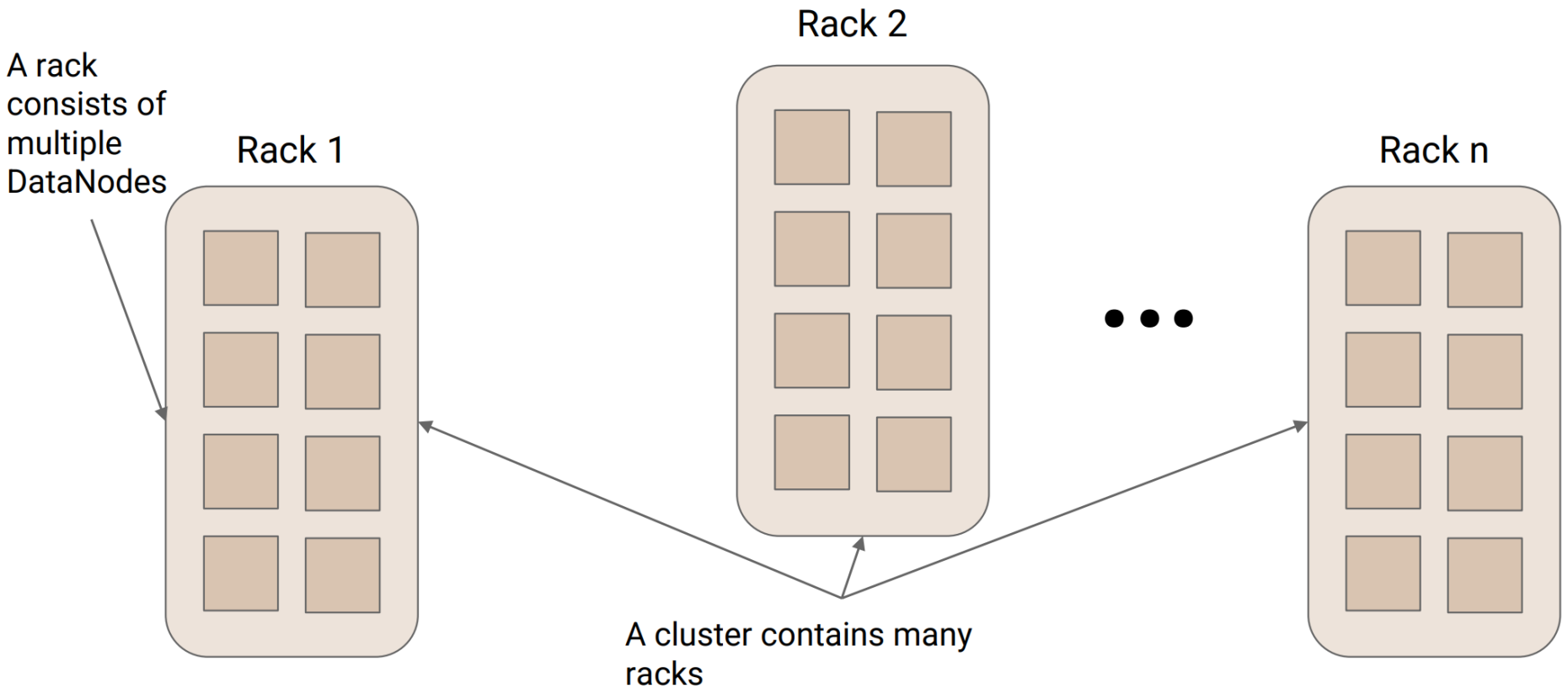
NameNode

- HDFS metadata is stored in the **NameNode**:
 - When the **file** was created, accessed, modified, deleted, and so on.
 - Where the **blocks** of the file are stored in the cluster.
 - Who has the **rights** to view or modify the file.
 - *How many files are stored on the cluster.*
 - *How many data nodes exist in the cluster.*
 - The **location of the transaction log** for the cluster.

Data nodes

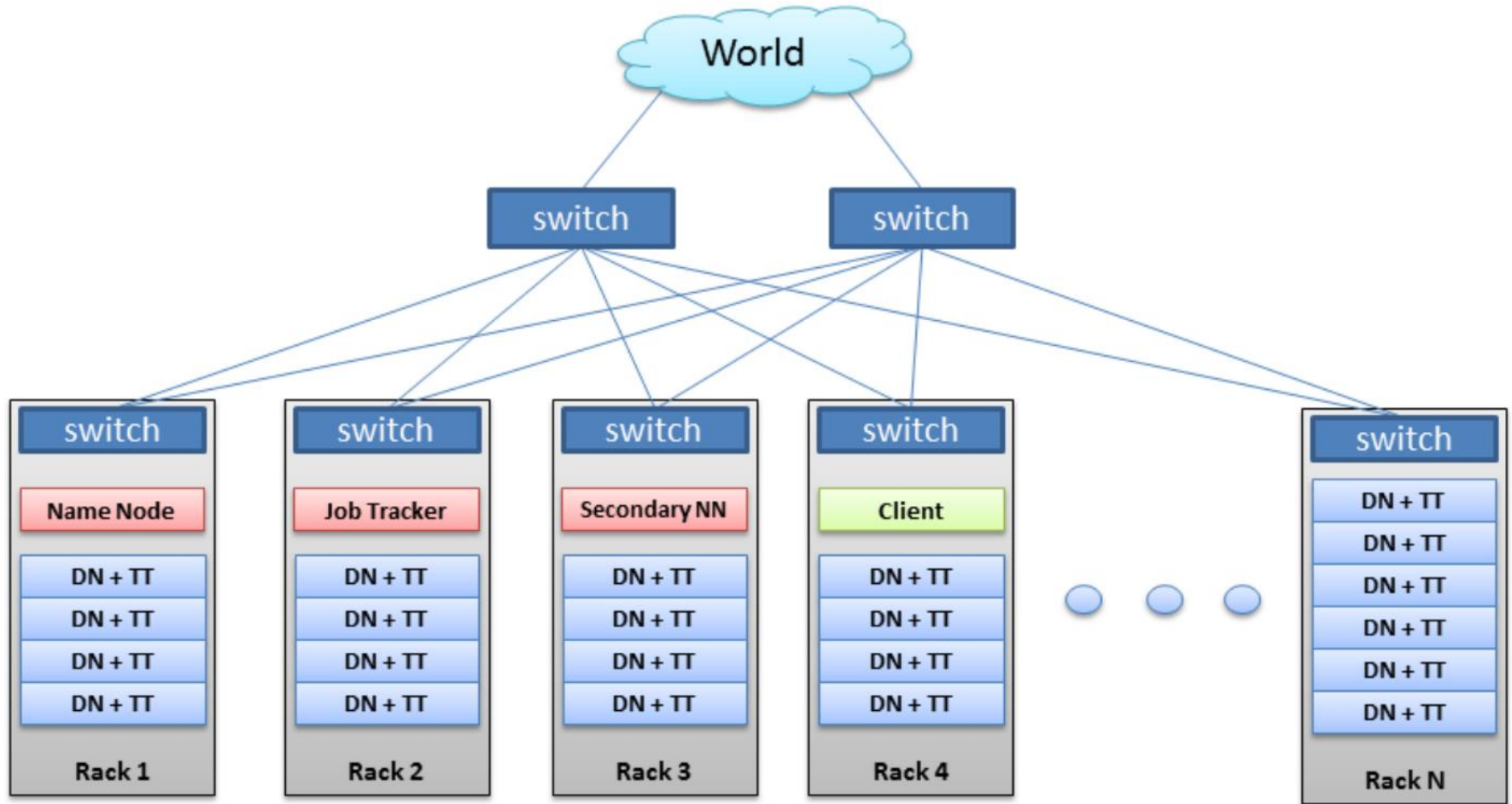
- **Data nodes** are servers that contain the **blocks** for a given set of files.
- It is reasonable to think of data nodes as “block servers”.
 - HDFS is available on many different operating systems and behaves the same whether on Windows, Mac OS, or Linux.
 - Stores (and retrieves) the data blocks in **the local file system of the server**.
 - Stores the metadata of a block in the local file system *based on the metadata template in the NameNode*.
 - Sends regular reports to the NameNode about what blocks are available for file operations.

Data nodes-what is a cluster?



Data nodes-what is a cluster?

Hadoop Cluster



BRAD HEDLUND .com

Possible Failures

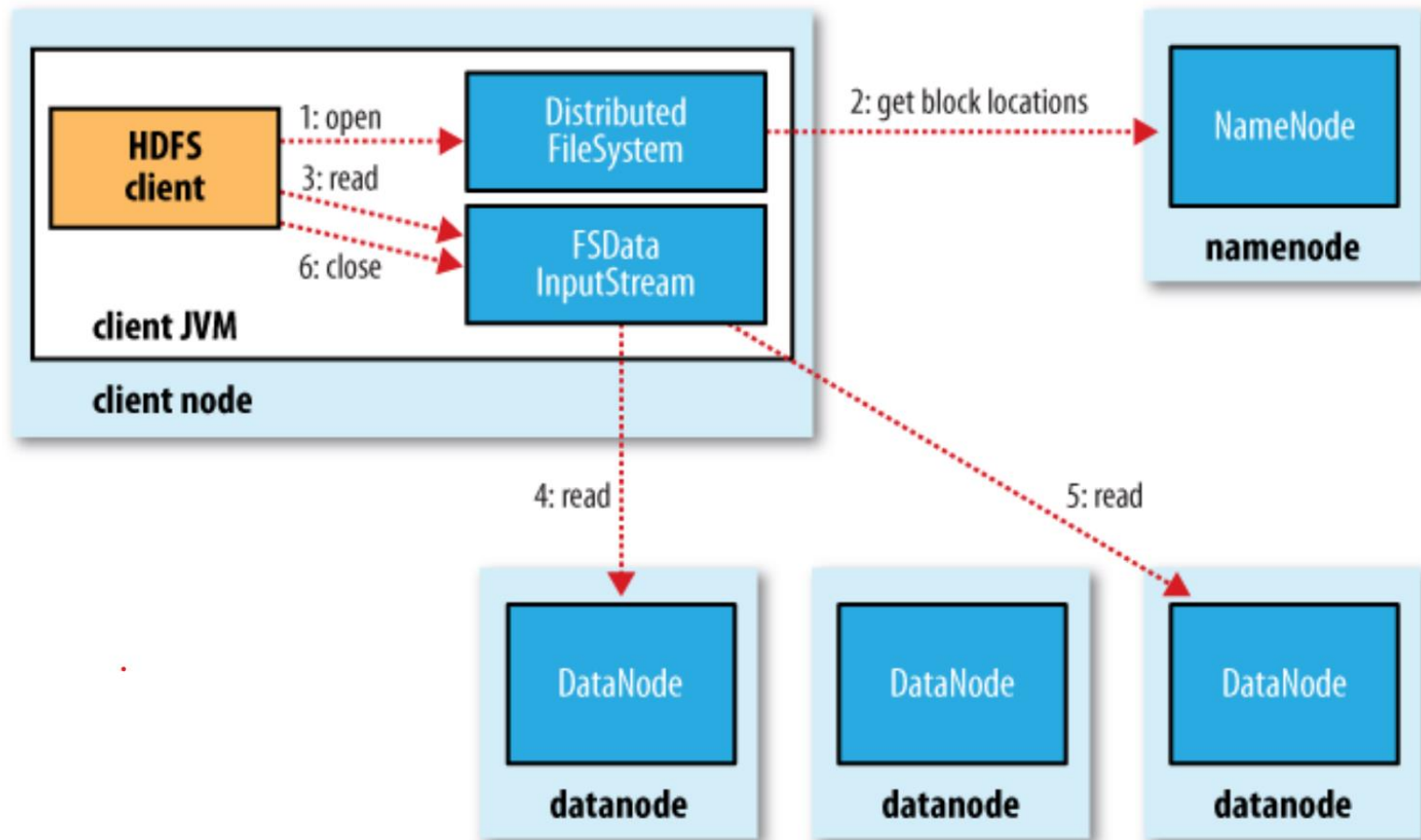
- The primary objective of HDFS is to store data reliably in the presence of failures.
- Three common failures that it must handle are:
 - DataNode failure
 - Network partition
 - NameNode failure

DataNode failure

- A crashed DataNode or a network partition can cause a subset of DataNodes to lose connectivity with the NameNode
- NameNode detects this by the absence of a heartbeat
 - NameNode marks these DataNodes, and does not send requests to them
 - Data registered to the failed DataNode is not available to the HDFS
 - Death of a DataNode may cause some blocks to require more replication

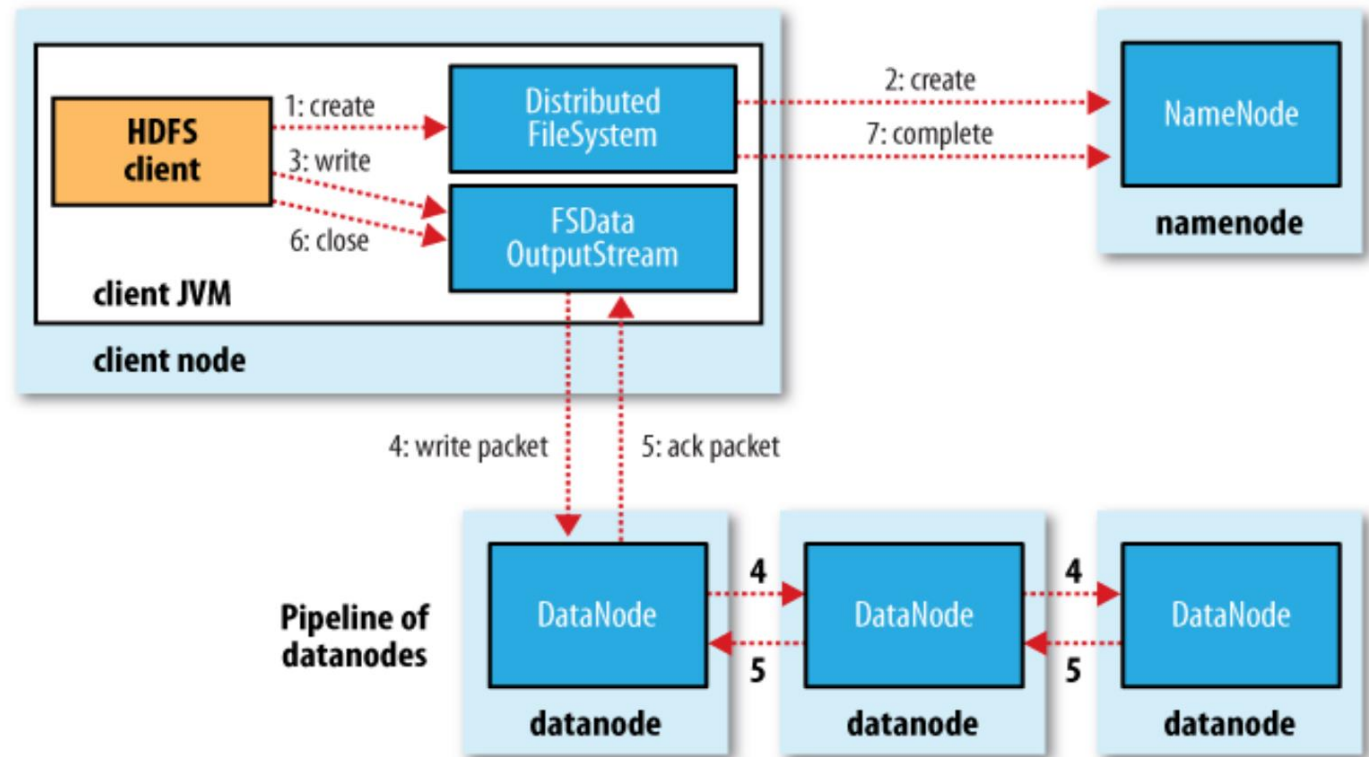
Client reading data from a HDFS

Client reading data from a HDFS



Ebook :Hadoop the definitive guide by tom white 3rd edition

Client writing data to HDFS



HDFS Characteristics

- **Scalable:** New nodes can be added as needed, without changing data formats, or how data is loaded, how jobs are written, or the applications on top.
- **Cost effective:** Hadoop brings massively parallel computing to commodity servers. The result is a sizeable decrease in the cost per terabyte of storage, which in turn makes it affordable to model all your data.
- **Flexible:** Hadoop is schema-less, and can absorb any type of data, structured or not, from any number of sources. Data from multiple sources can be joined and aggregated in arbitrary ways enabling deeper analyses than any one system can provide.
- **Fault tolerant:** When you lose a node, the system redirects work to another location of the data and continues processing without missing a beat

Example

File Edit View Search Terminal Help

```
[training@localhost data]$ hadoop fs -mv purchases.txt newname.txt
[training@localhost data]$ hadoop fs -ls
Found 1 items
-rw-r--r--    1 training supergroup  211312924 2013-09-12 21:16 newname.txt
[training@localhost data]$ hadoop fs -rm newname.txt
Deleted newname.txt
[training@localhost data]$ hadoop fs -mkdir myinput
[training@localhost data]$ hadoop fs -put purchases.txt myinput
[training@localhost data]$ hadoop fs -ls
Found 1 items
drwxr-xr-x    - training supergroup           0 2013-09-12 21:16 myinput
[training@localhost data]$ hadoop fs -ls myinput
Found 1 items
-rw-r--r--    1 training supergroup  211312924 2013-09-12 21:16 myinput/purchases.txt
[training@localhost data]$ █
```

Example

```
File Edit View Search Terminal Help
[training@localhost data]$ ls
access_log.gz  purchases.txt
[training@localhost data]$ hadoop fs -ls
[training@localhost data]$ hadoop fs -put purchases.txt
[training@localhost data]$ hadoop fs -ls
Found 1 items
-rw-r--r-- 1 training supergroup 211312924 2013-09-12 21:16 purchases.txt
[training@localhost data]$ hadoop fs -tail purchases.txt
31      17:59  Norfolk Toys      164.34  MasterCard
2012-12-31      17:59  Chula Vista      Music    380.67  Visa
2012-12-31      17:59  Hialeah Toys     115.21  MasterCard
2012-12-31      17:59  Indianapolis     Men's Clothing 158.28  MasterCard
2012-12-31      17:59  Norfolk Garden   414.09  MasterCard
2012-12-31      17:59  Baltimore        DVDs     467.3   Visa
2012-12-31      17:59  Santa Ana        Video Games 144.73  Visa
2012-12-31      17:59  Gilbert Consumer Electronics 354.66  Discover
2012-12-31      17:59  Memphis Sporting Goods 124.79  Amex
2012-12-31      17:59  Chicago Men's Clothing 386.54  MasterCard
2012-12-31      17:59  Birmingham       CDs      118.04  Cash
2012-12-31      17:59  Las Vegas        Health and Beauty 420.46  Amex
2012-12-31      17:59  Wichita Toys     383.9   Cash
2012-12-31      17:59  Tucson Pet Supplies 268.39  MasterCard
2012-12-31      17:59  Glendale         Women's Clothing 68.05   Amex
2012-12-31      17:59  Albuquerque      Toys     345.7   MasterCard
2012-12-31      17:59  Rochester        DVDs     399.57  Amex
2012-12-31      17:59  Greensboro       Baby     277.27  Discover
2012-12-31      17:59  Arlington        Women's Clothing 134.95  MasterCard
2012-12-31      17:59  Corpus Christi   DVDs     441.61  Discover
[training@localhost data]$
```

MapReduce



Introduction

- MapReduce is a programming model for Big Data **processing**.
- MapReduce is designed to efficiently process large volumes of data by connecting many distributed commodity computers together to work in parallel.
- The power of MapReduce lies in its ability to scale to 100s and 1000s of machines, each with several processor cores.

Google

MapReduce is a programming model that **developers at Google** designed as a solution for handling massive amounts of search data. Worldwide, the amount of data we produce has exploded in recent years, with projected data use for 2025 estimated to be over 180 zettabytes [1]. Mar 22, 2024



Coursera

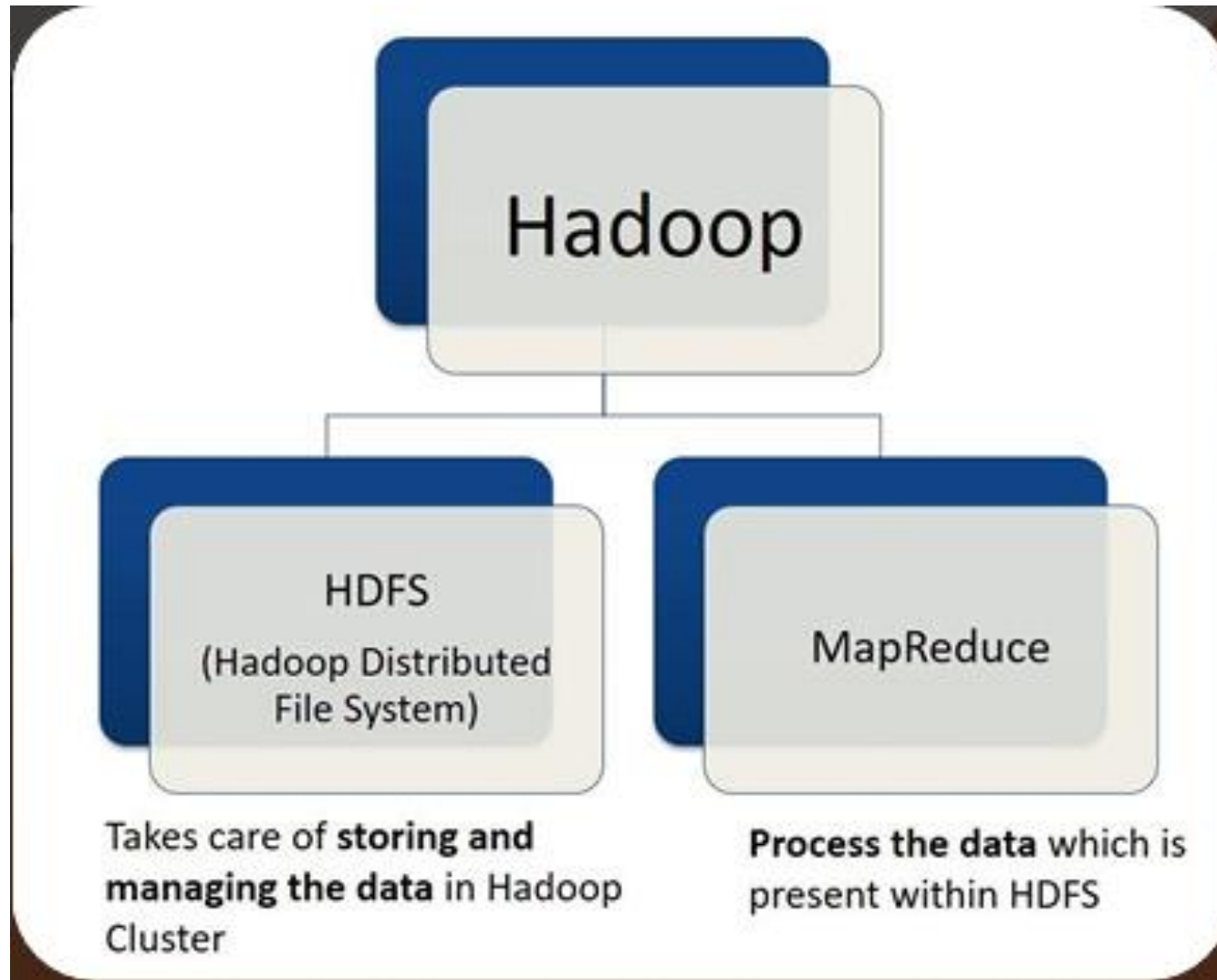
<https://www.coursera.org> › ... › Data › Data Analytics

What Is MapReduce? - Coursera

Introduction

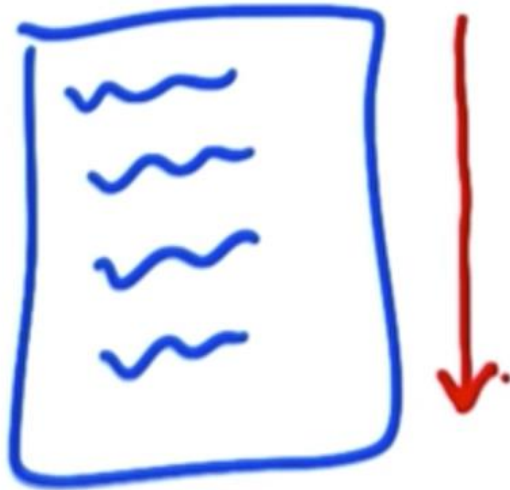
- Professionals across many industries use MapReduce, but it was popularized through its use at industry giants such as **Google, Yahoo, and Facebook.**
- Professionals at Google use MapReduce to cluster articles, create indexes on Google Search (PageRank), compute street maps, and in machine translation, ...
- Yahoo similarly uses MapReduce for map functions and spam detection, ...
- Facebook uses MapReduce for data mining and analytics, spam detection, and advertisement strategy, ...

Introduction



Problem

- Processing large files from top to bottom could take **long time** and could be **out of memory**.



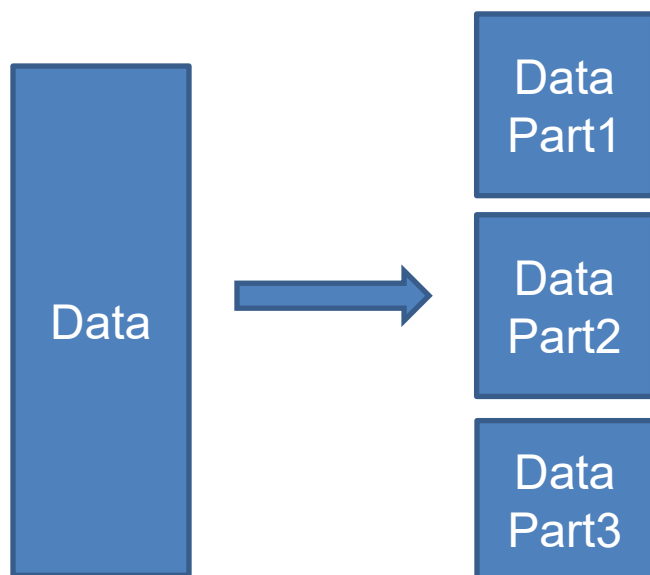
Fast parallel computation

- MapReduce enables parallel processing by partitioning vast datasets into smaller segments, processing them concurrently across Hadoop commodity servers.
- Ultimately, it consolidates the results from multiple servers to deliver a unified output to the application.
- For example, consider a Hadoop cluster comprising 20,000 cost-effective commodity servers, each handling a **256MB** blocks of data.
- This configuration allows the concurrent processing of approximately **5TB** of data, significantly decreasing the processing time compared to the sequential processing of such a substantial dataset.

Basic idea of MapReduce

Phase 1 (Map)

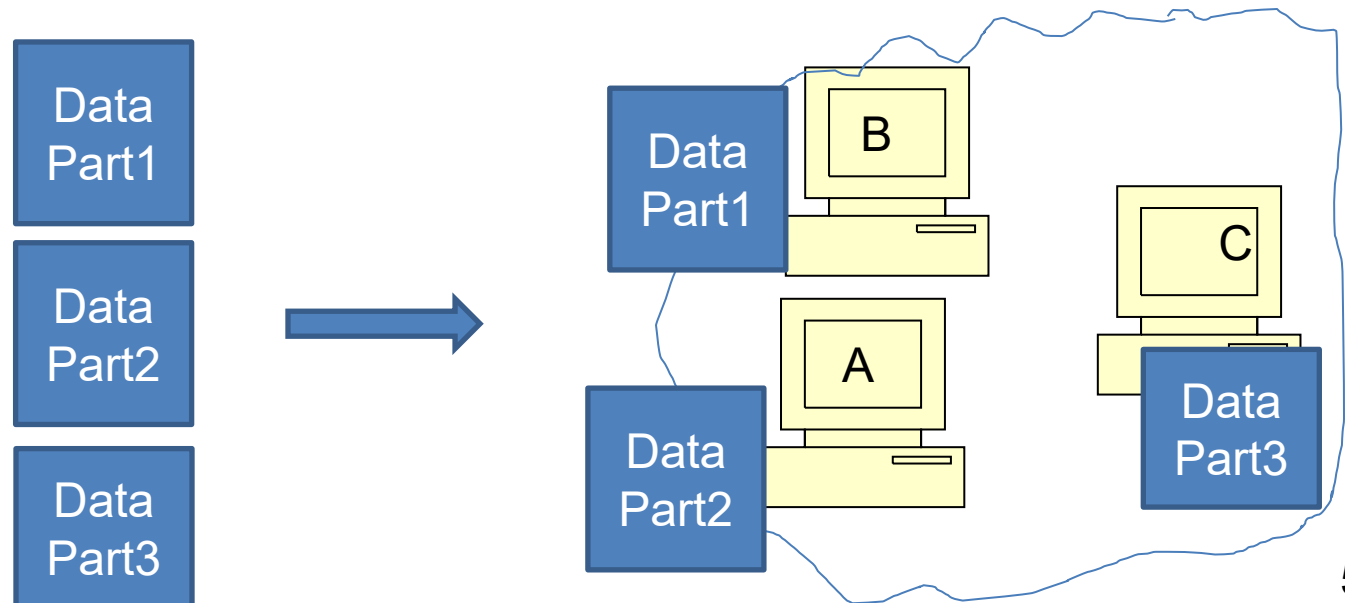
1. Split the data into blocks



Basic idea of MapReduce

Phase 1 (Map)

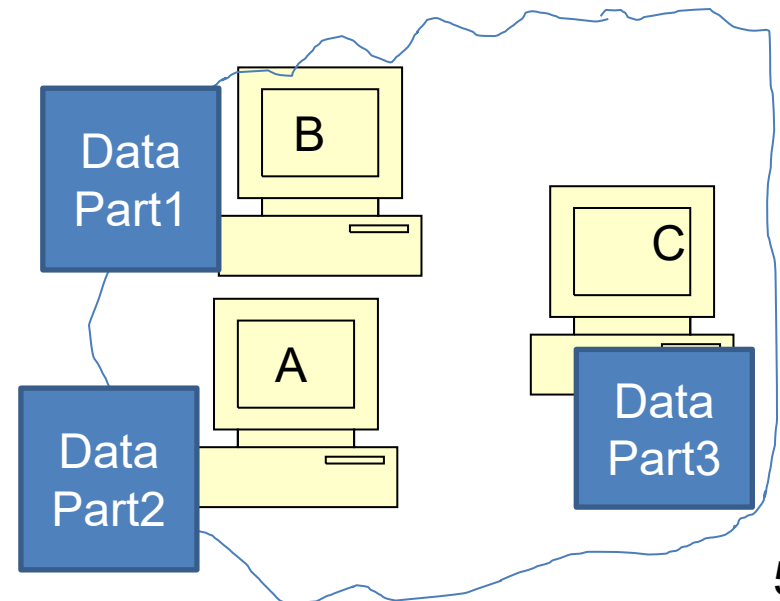
1. Split the data into blocks
2. **Assign each block to an instance (e.g. a computer or virtual machine)**



Basic idea of MapReduce

Phase 1 (Map)

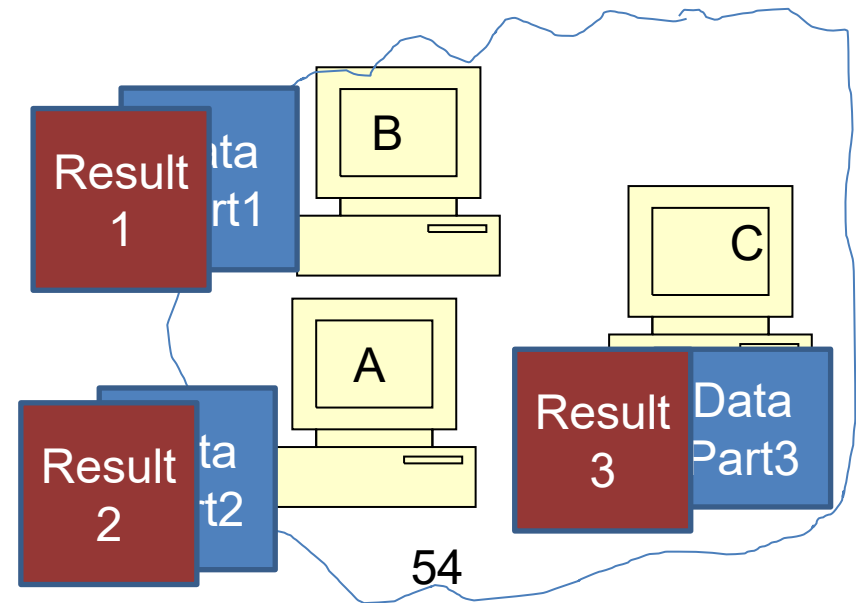
1. Split the data into blocks
2. Assign each block to an instance (e.g. a computer or virtual machine)
3. Run these instances in parallel



Introduction

Phase 2 (Reduce)

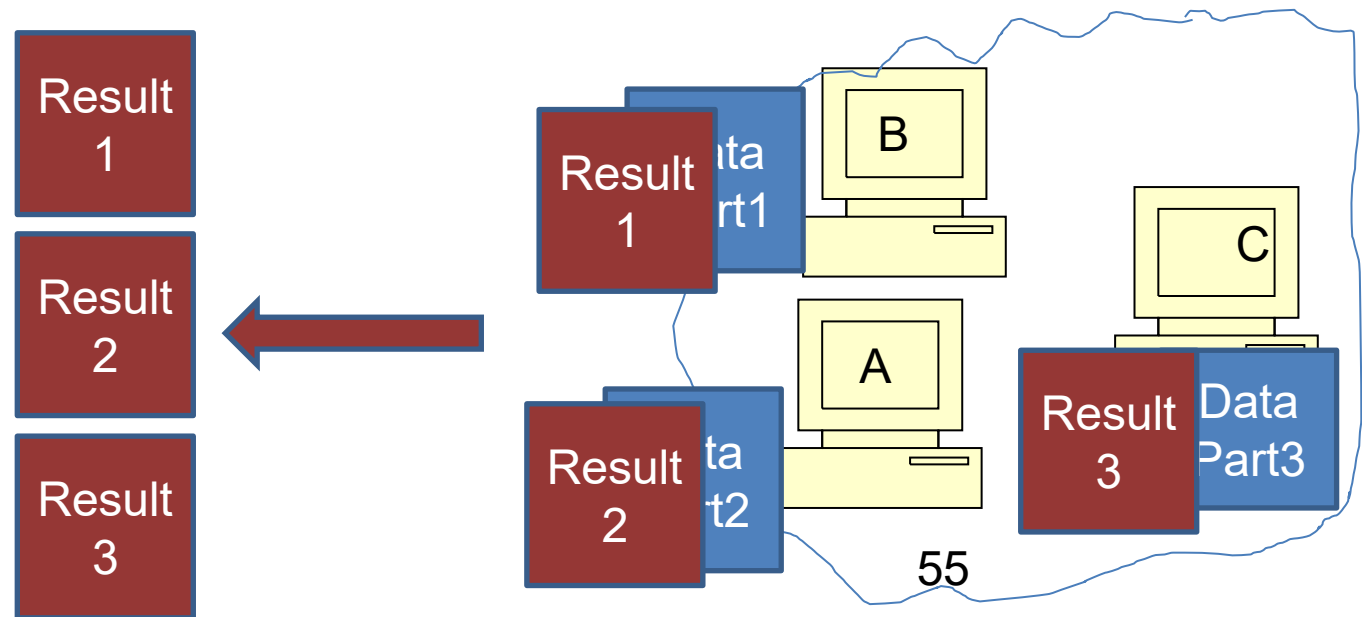
1. Once all the instances have finished their sub-tasks,



Introduction

Phase 2 (Reduce)

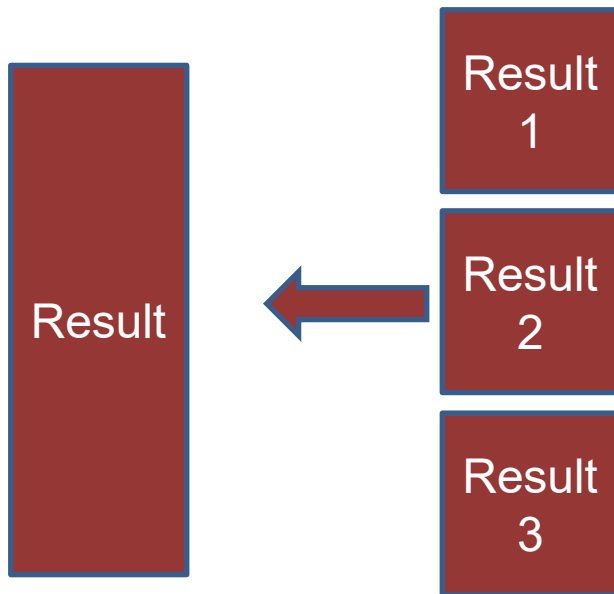
1. Once all the instances have finished their sub-tasks, **they send their results.**



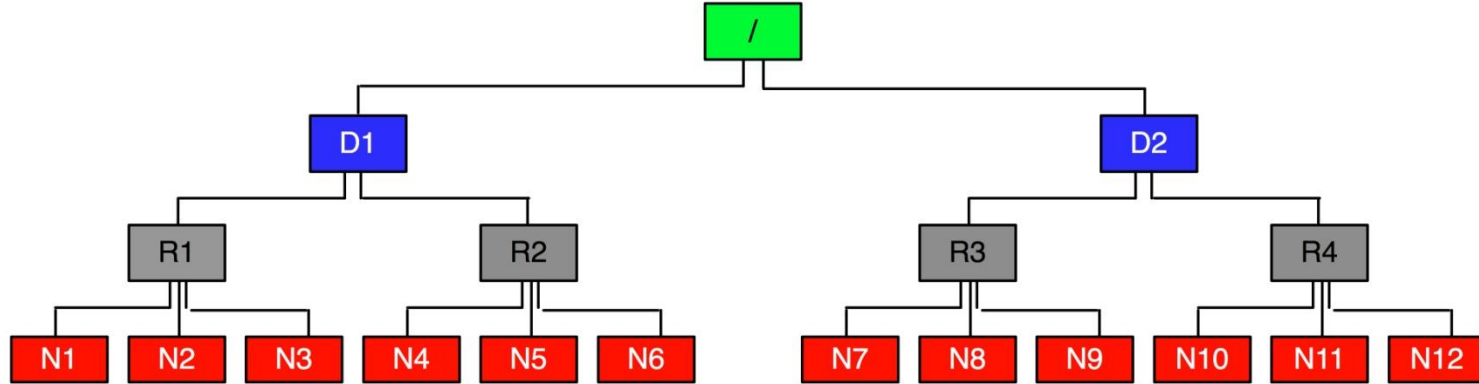
Introduction

Phase 2 (Reduce)

1. Once all the instances have finished their sub-tasks, they send their results.
2. **Results are merged to obtain the final result.**



Cluster Network



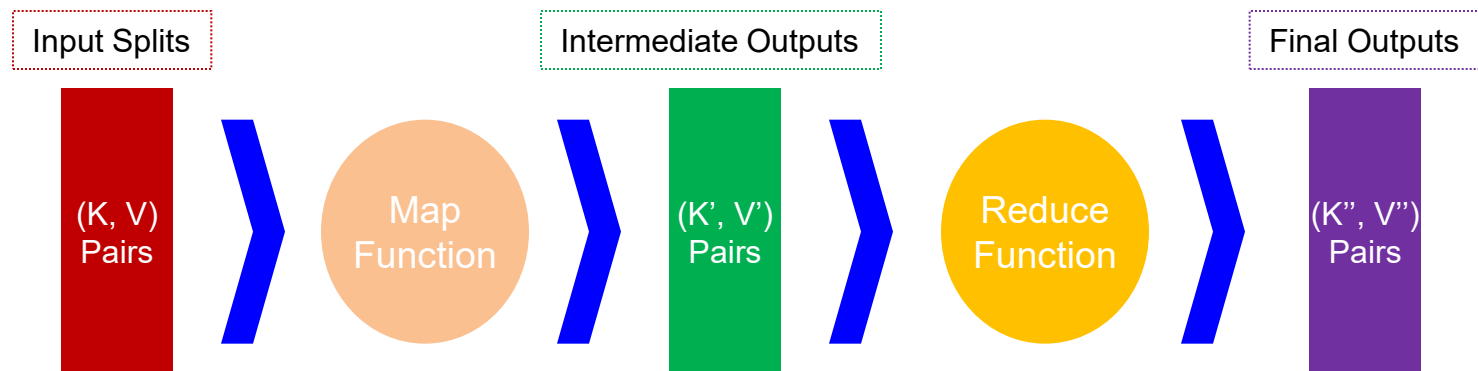
- MapReduce assumes a tree-style, master-slave network topology
- Nodes are spread over different *racks* contained in one or many data centers
- *A salient point is that the bandwidth between two nodes is dependent on their relative locations in the network topology*
- For example, **nodes that are on the same rack will have higher bandwidth between them as opposed to nodes that are off-rack**

Computing Units: Tasks

- MapReduce divides the workload into multiple independent **tasks** and automatically schedule them on cluster nodes.
- A work performed by each task is done in isolation from one another.
- Communication between tasks is limited for scalability and fault tolerance.

Data Structure: Keys and Values

- The programmer in MapReduce has to specify two functions, the Map function and the Reduce function that implement the Mapper and the Reducer in a MapReduce program.
- In MapReduce, data elements are always structured as key-value (i.e., (K, V)) pairs.
- The Map and Reduce functions receive and emit (K, V) pairs.



A simple example 1

- Consider that we want to **count** how many times each word appear in a very large text document.

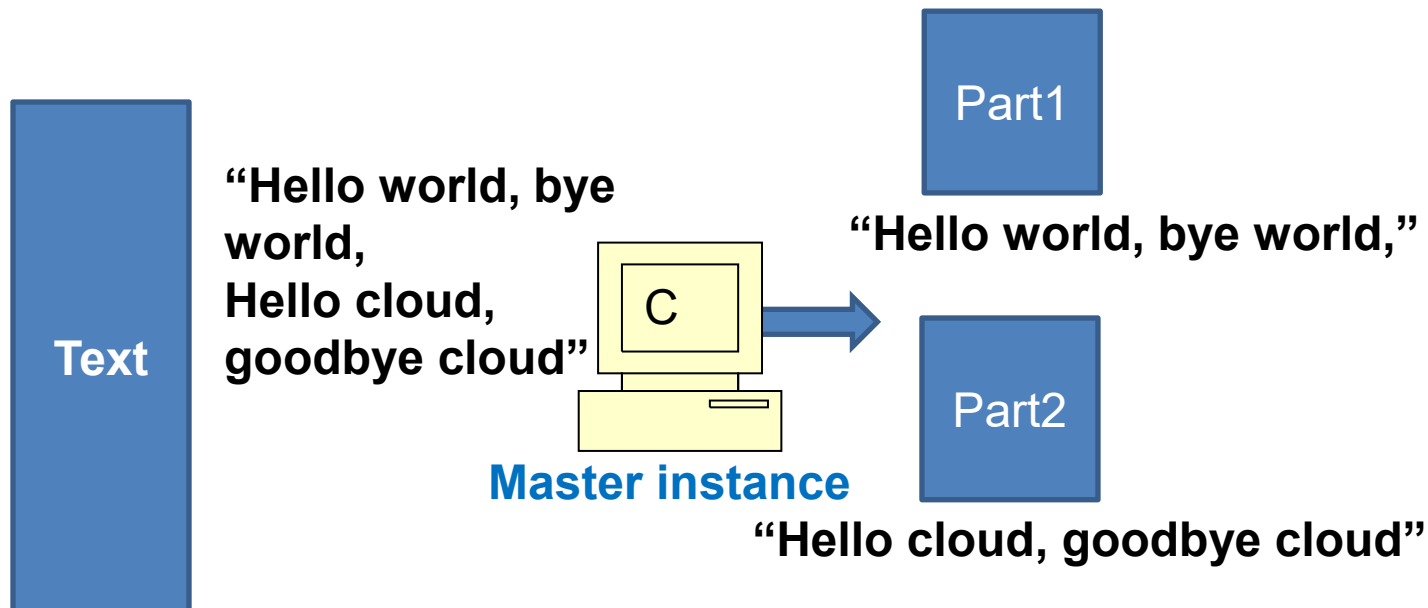


Text

“Hello world, bye
world,
Hello cloud,
goodbye cloud”

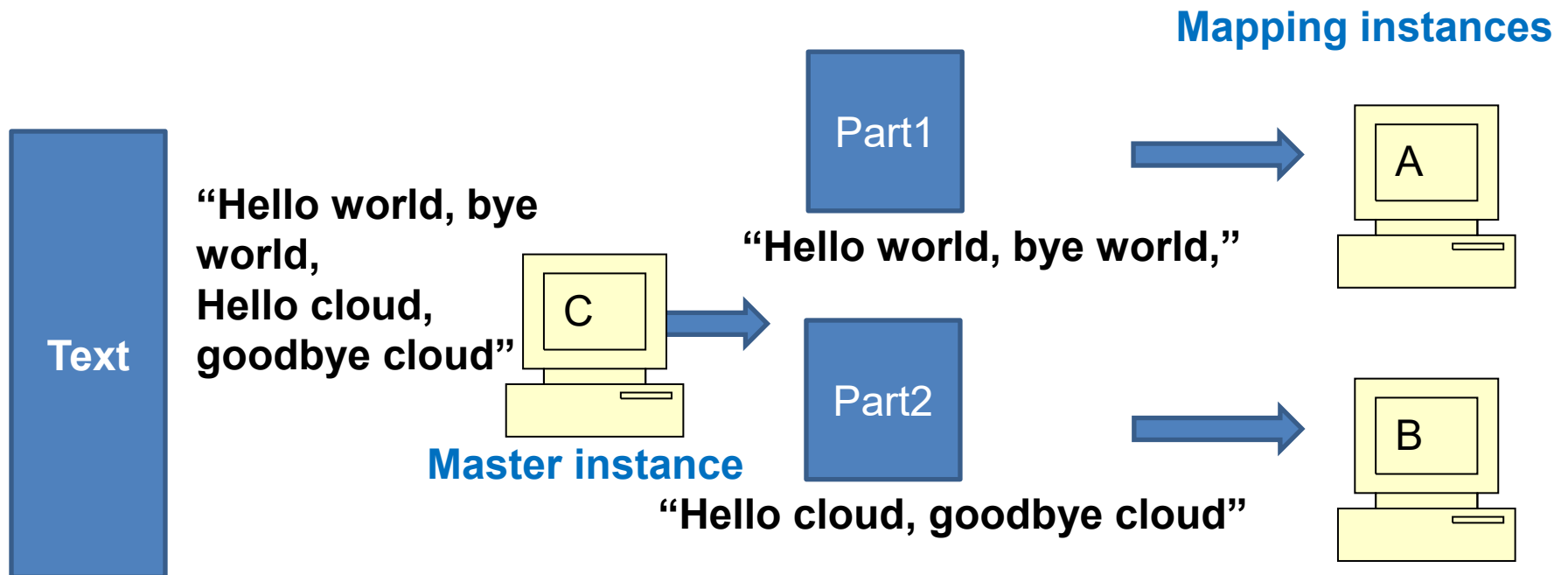
A simple example 1

- The **master** instance first splits the data into **M data blocks**. (i.e. HDFS)



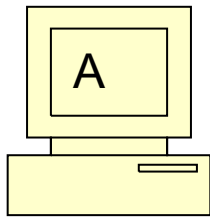
A simple example 1

- Then, it starts M mapping instances and gives a data block to each instance. (i.e. HDFS)



A simple example 1 – map

- **All instances work in parallel.**
- **Consider the first instance.** It reads its data.
- It creates a **<key,value>** pair for each word that it reads. A **key** is a word and the corresponding **value** is the number 1.



“Hello world, bye world,”

<Hello, 1>

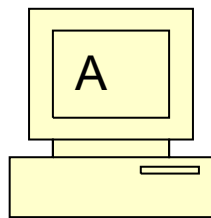
<World, 1>

<Bye, 1>

<World, 1>

A simple example 1 – map

- Some words like “World” appear multiple times in the result.
- All values that have the same key are grouped together.



“Hello world, bye world,”

<Hello, 1>

<World, 1>

<Bye, 1>

<World, 1>



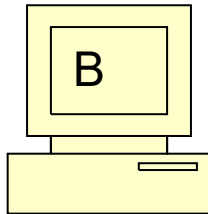
<Hello, 1>

<World, 2>

<Bye, 1>

A simple example 1 – map

- Consider the second instance.
- The second instance reads its data.
- It creates a `<key,value>` pair for each word that it reads. A `key` is a word and the corresponding `value` is the number 1.



“Hello cloud, goodbye cloud”

`<Hello, 1>`

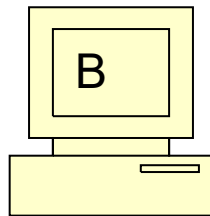
`<Cloud, 1>`

`<Goodbye, 1>`

`<Cloud, 1>`

A simple example 1 – map

Then, the **second instance** groups all values that have the same key together.



“Hello cloud, goodbye cloud”

<Hello, 1>

<Cloud, 1>

<Goodbye, 1>

<Cloud, 1>



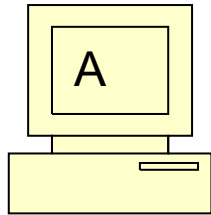
<Hello, 1>

<Cloud, 2>

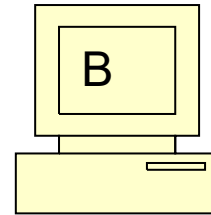
< Goodbye, 1>

A simple example 1

- So until now, we have:



<Hello, 1>
<World, 2>
<Bye, 1>

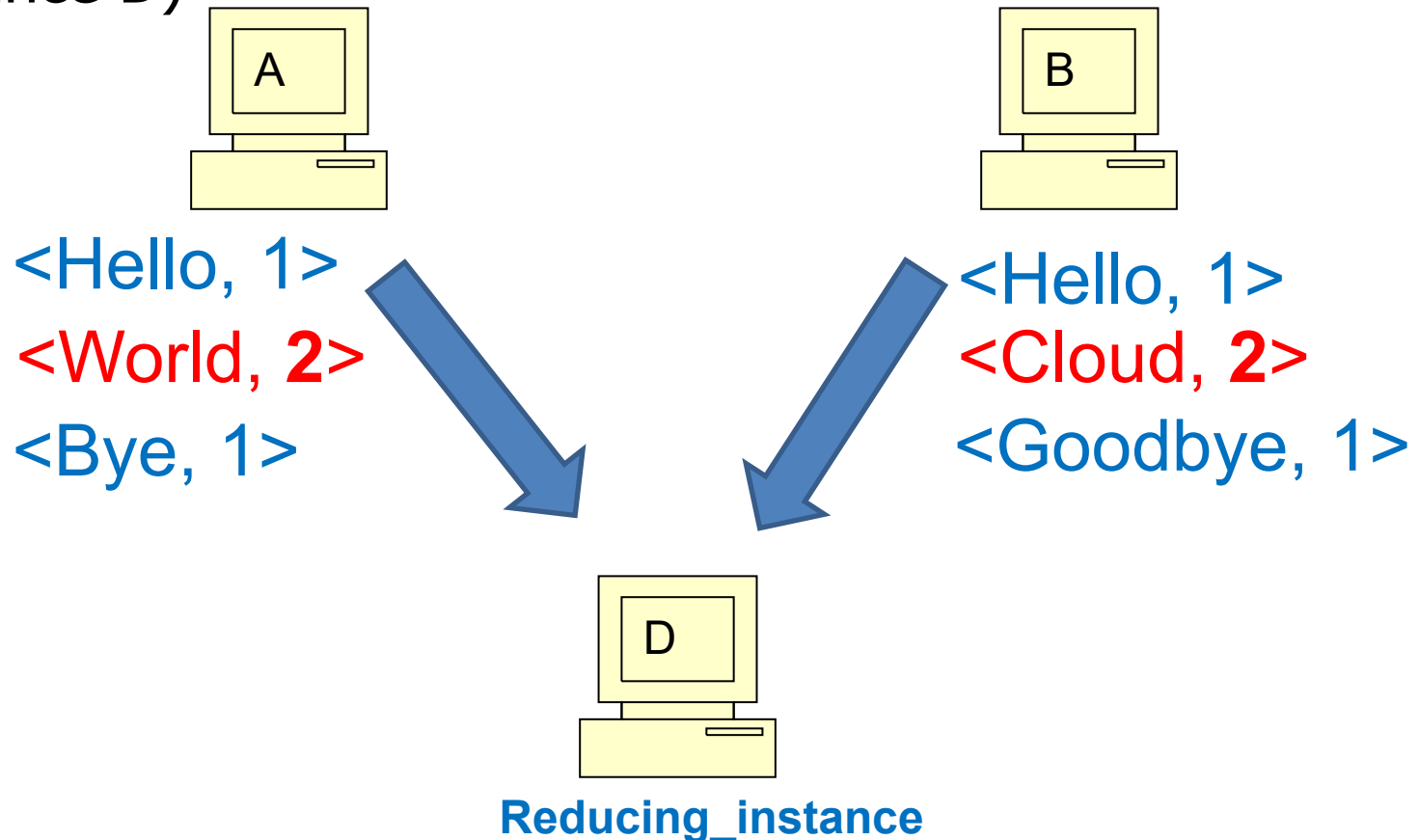


<Hello, 1>
<Cloud, 2>
<Goodbye, 1>

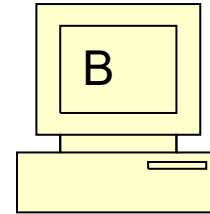
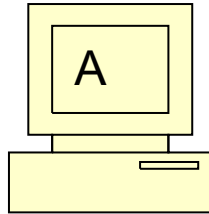
Next, the **reduce phase** will combine the local results found by all instances.

A simple example 1 – reduce

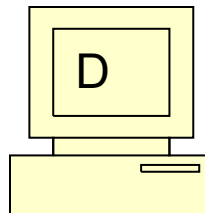
- The **master instance** will start **R** reducing instances for combining results of mapping instances.
- In this example, only one reducing instance is used (instance D)



A simple example 1 – reduce



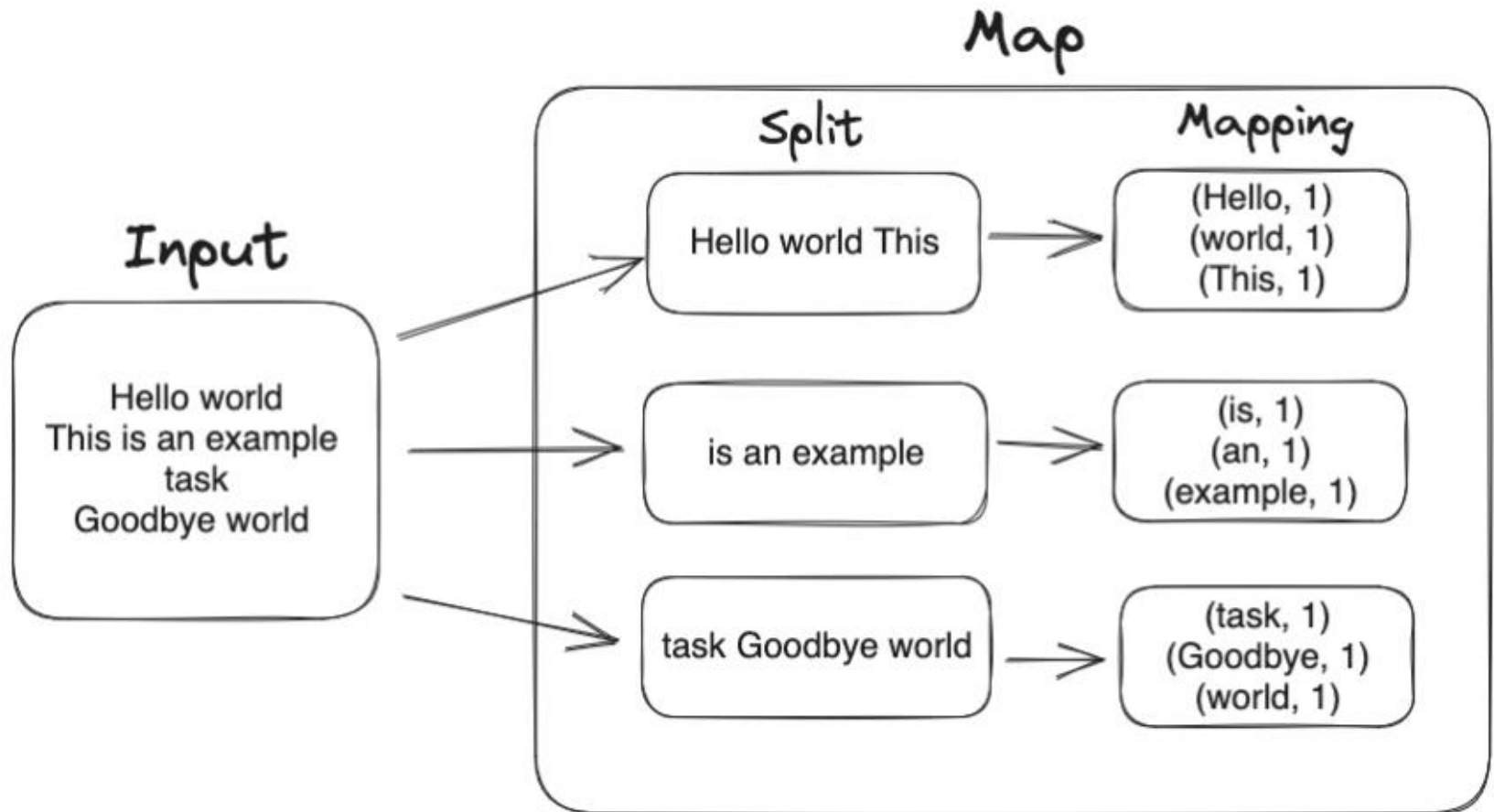
**This is the
final result!**



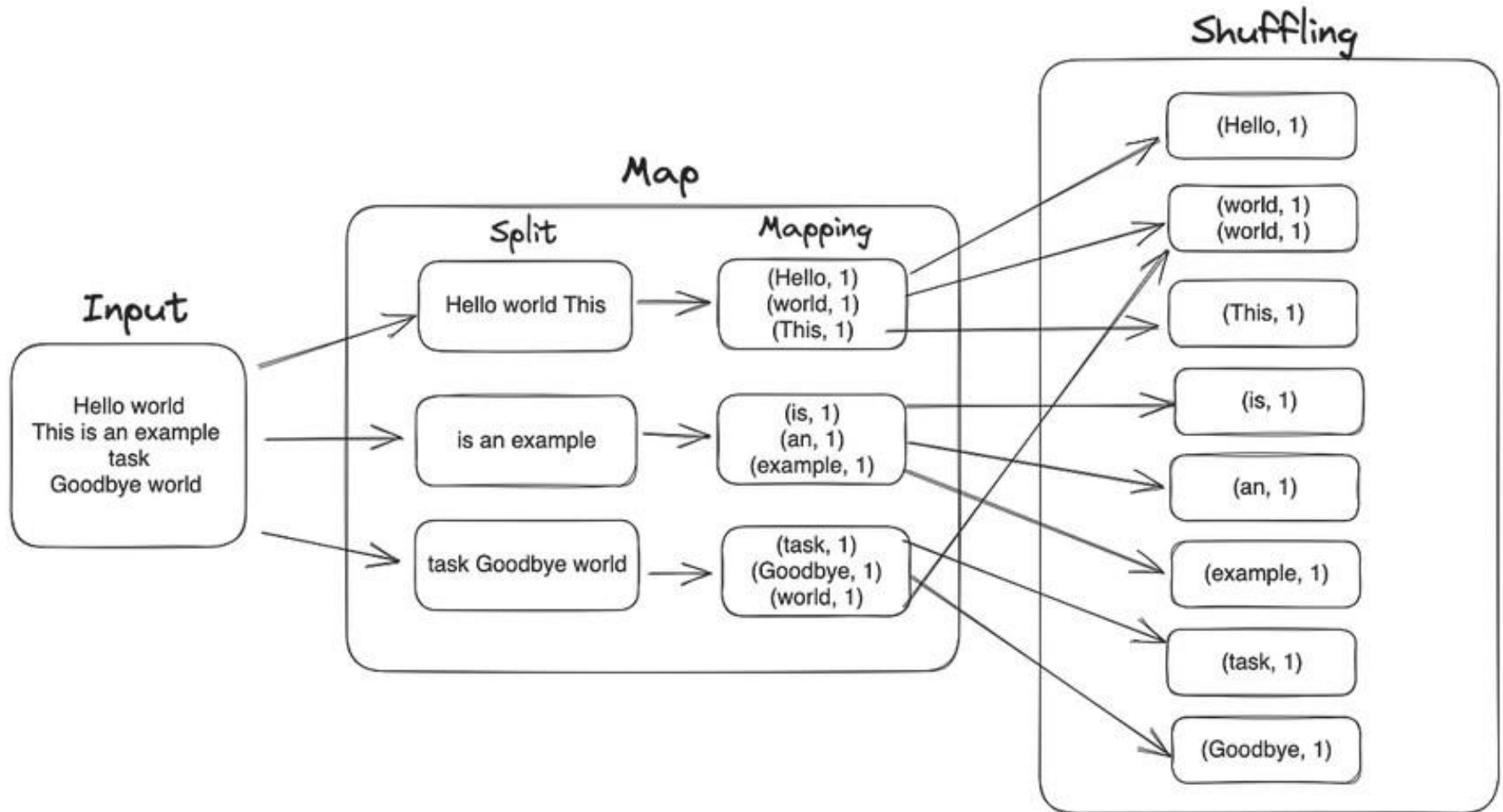
Reducinginstance

<Hello, 2>
<Cloud, 2>
<World, 2>
<Bye, 1>
<Goodbye, 1>

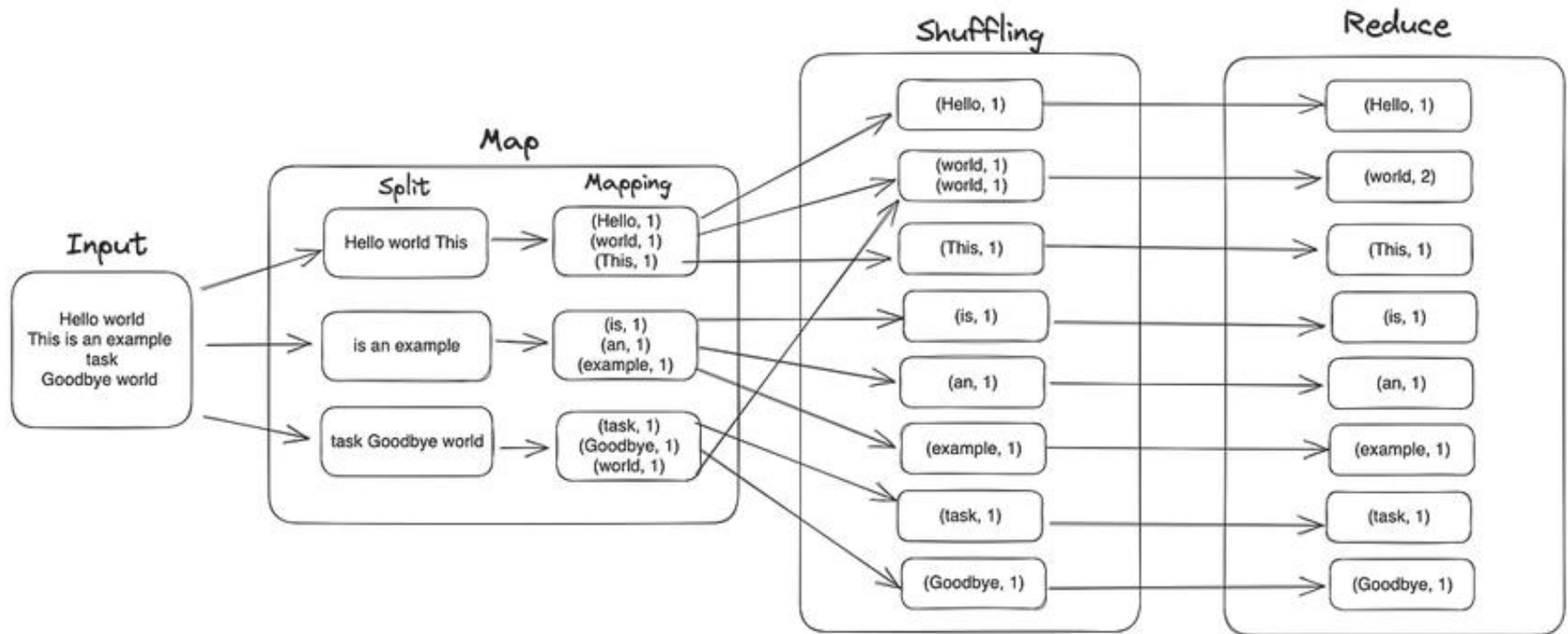
A simple example 2



A simple example 2



A simple example 2



A simple example 3

A Chunk of File

Mohammad is delivering a lecture to the 15-440 class

A Map Function

Key1	Value1
0	Mohammad is
20	delivering a
38	lecture to the
60	15-440 class

Parse & Count

Key2	Value2
Mohammad	1
is	1
delivering	1
a	1
lecture	1
to	1
the	1
15-440	1
class	1

A Chunk of File

The course name of 15-440 is Distributed Systems

A Map Function

Key1	Value1
0	The course
17	name of 15-440
40	is Distributed
58	Systems

Parse & Count

Key2	Value2
The	1
course	1
name	1
of	1
15-440	1
is	1
Distributed	1
Systems	1

A Reduce Function

Iterate & Sum

Key2	Value2
Mohammad	1
is	2
delivering	1
a	1
lecture	1
to	1
the	2
15-440	2
class	1
course	1
name	1
of	1
Distributed	1
Systems	1

A simple example 4

- Select maximum temperature for each date
 - Input: two structured textual files containing the temperatures gathered by a set of sensors
 - Each line of the first file has the following format
sensorID,date,hour,temperature\n
 - Each line of the second file has the following format
date,hour,temperature,sensorID\n
 - Output: the maximum temperature for each date
(considering the data of both input files)

- Input files

```
s1,2016-01-01,14:00,20.5
s2,2016-01-01,14:00,30.2
s1,2016-01-02,14:10,11.5
s2,2016-01-02,14:10,30.2
```

```
2016-01-01,14:00,20.1,s3
2016-01-01,14:00,10.2,s4
2016-01-02,14:15,31.5,s3
2016-01-02,14:15,20.2,s4
```

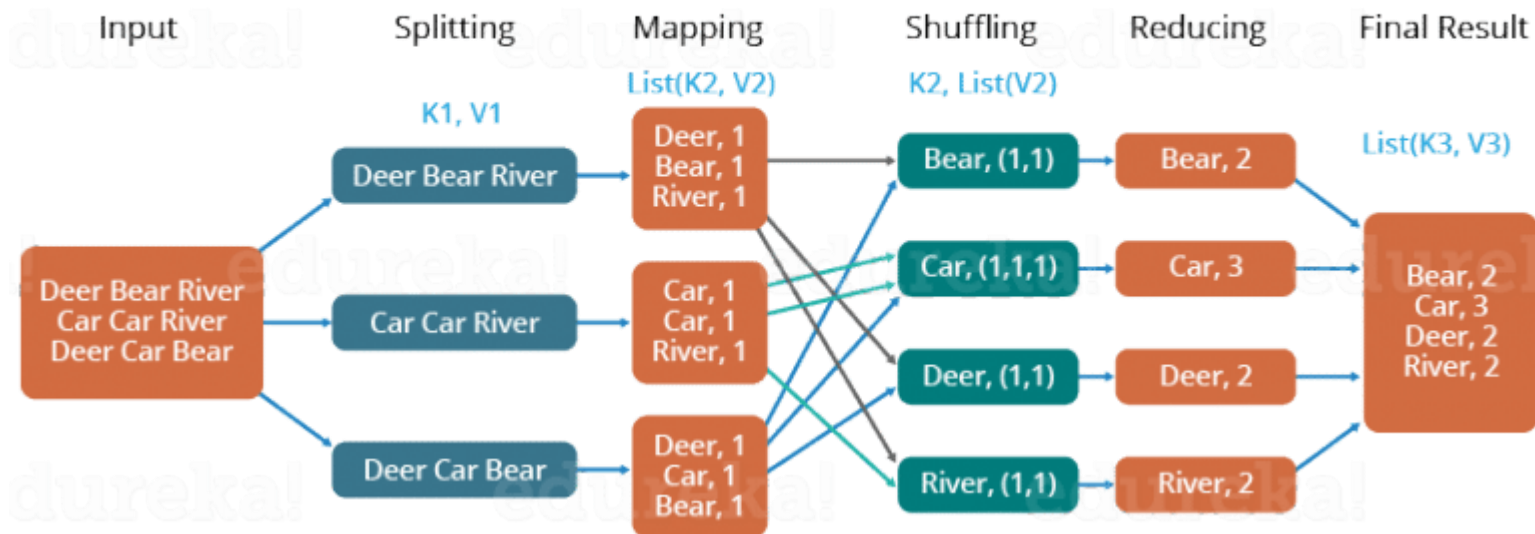
- Output

2016-01-01	30.2
2016-01-02	31.5

A simple example 5

The Overall MapReduce Word Count Process

edureka!



A simple example 6

Scenario



- Let's imagine we run a retailer with thousands of stores around the world, and we have a big ledger that contains all of our sales, the big book of sales for the year 2012

A simple example 6

Scenario



Total sales per store

- We've been asked to calculate things like the total sales per store for the year

A simple example 6

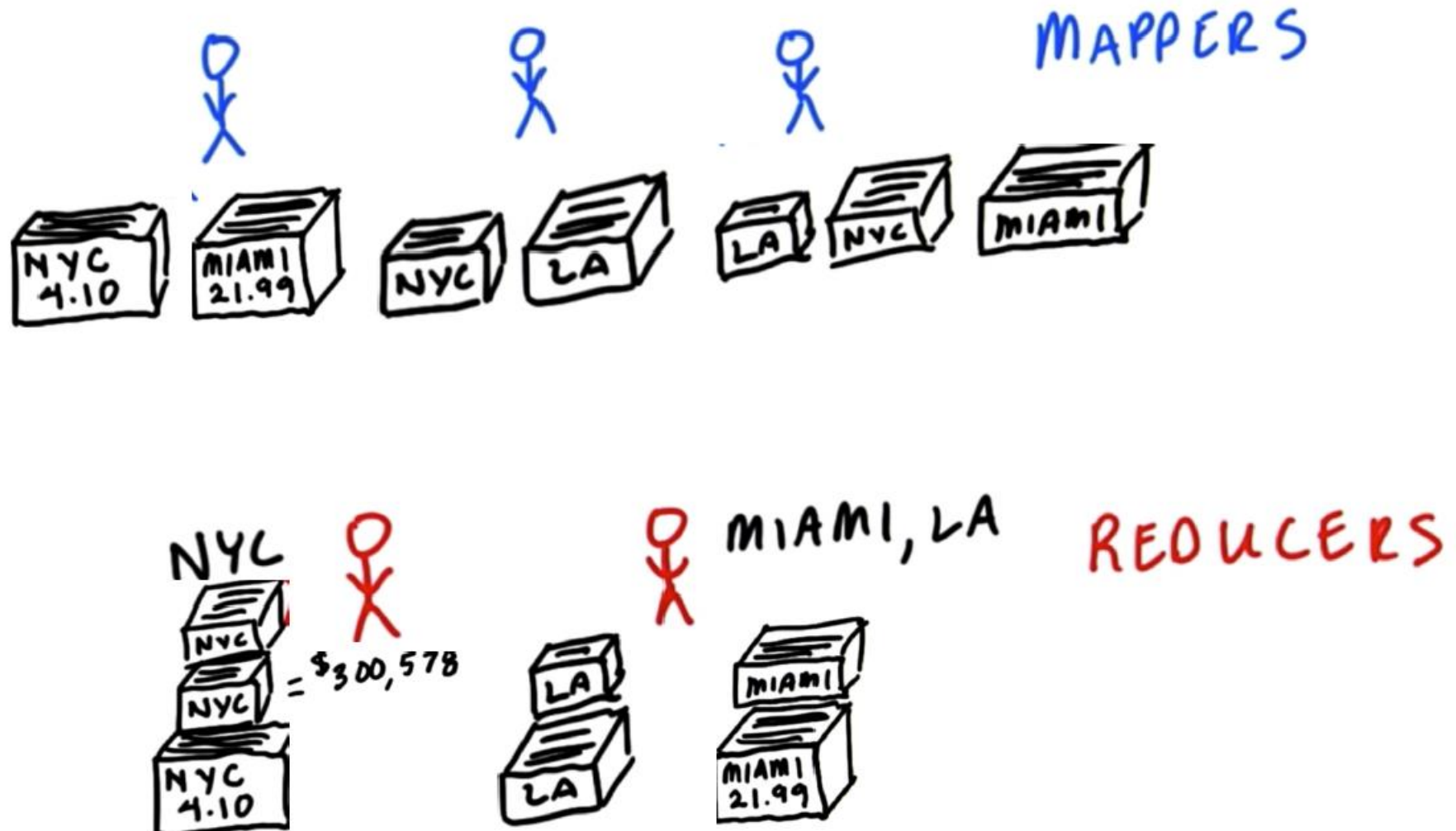
Scenario



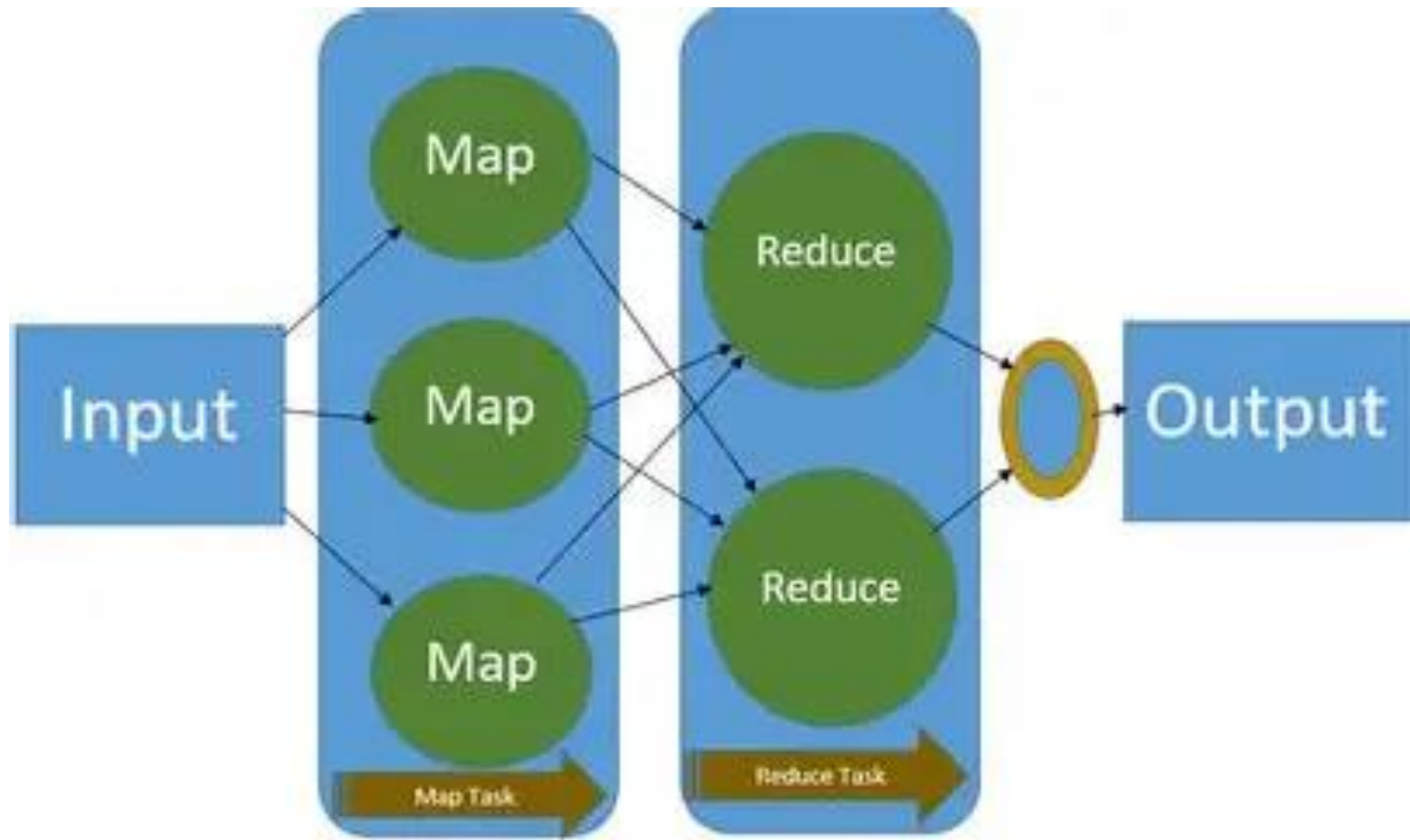
2012-01-01 London Clothes 25.99
2012-01-01 Miami Music 12.15
2012-01-02 NYC Toys 3.10
2012-01-02 Miami Clothes 50.00

- The big book contains: The date of the purchase, the location of the store, what they bought, and the amount

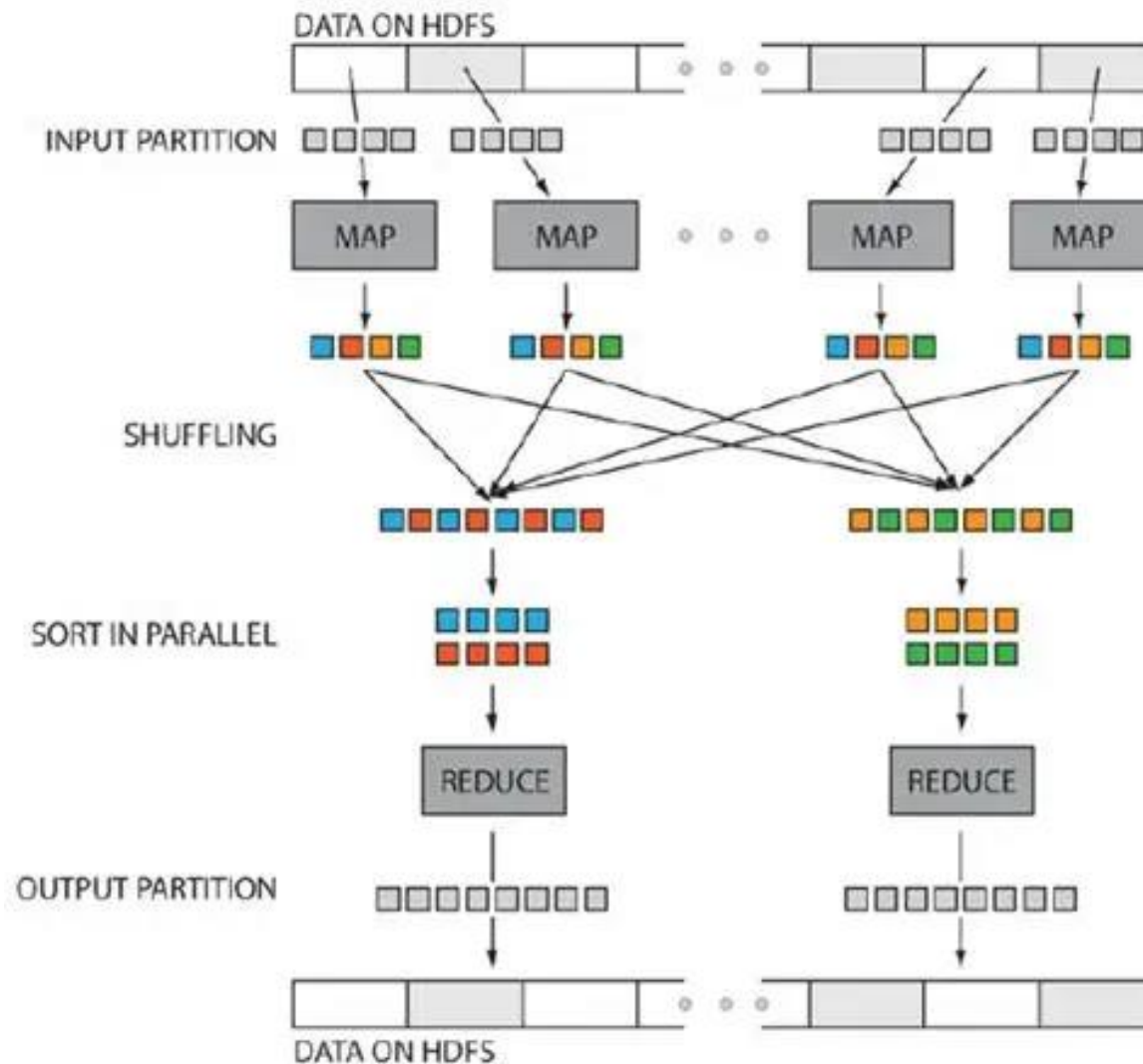
A simple example 6



Basic Architecture

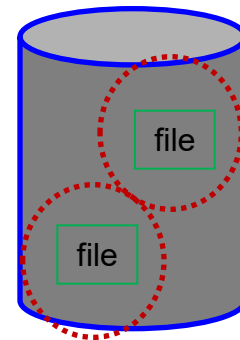


Detailed Architecture



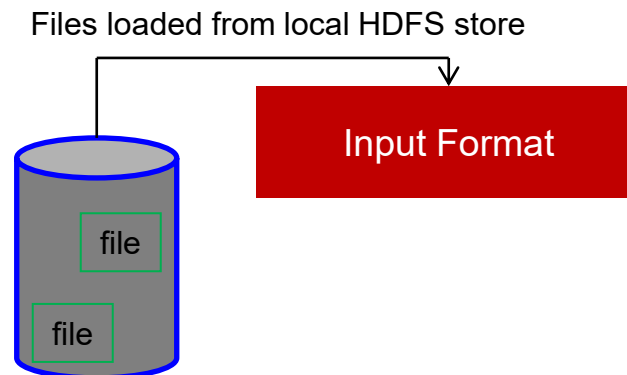
Input Files

- Input files are where the data for Map tasks are initially stored
- The input files typically reside in a distributed file system (e.g. HDFS)
- The format of input files is arbitrary
 - Line-based log files
 - Binary files
 - Multi-line input records
 - Or something totally different



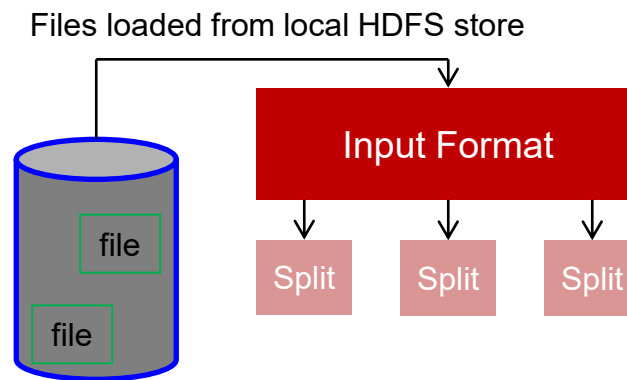
Input Format

- **Input Format** defines How the input files are split up and read.
- Input Format is a Hadoop **class** that does the following:
 - Selects the files that should be used for input.
 - Defines the Input Splits that break a file.



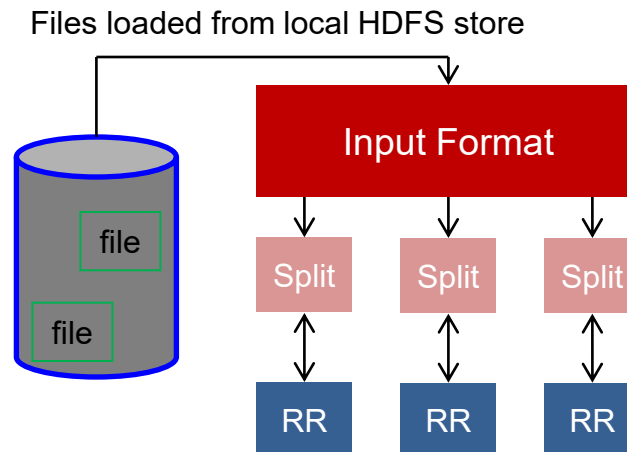
Input Splits

- An **input split** describes a unit of work that comprises a single Map task in a MapReduce program.
- By default, the Input Format breaks a file up into 64MB splits.
- By dividing the file into splits, we allow several Map tasks to operate on a single large file in parallel.



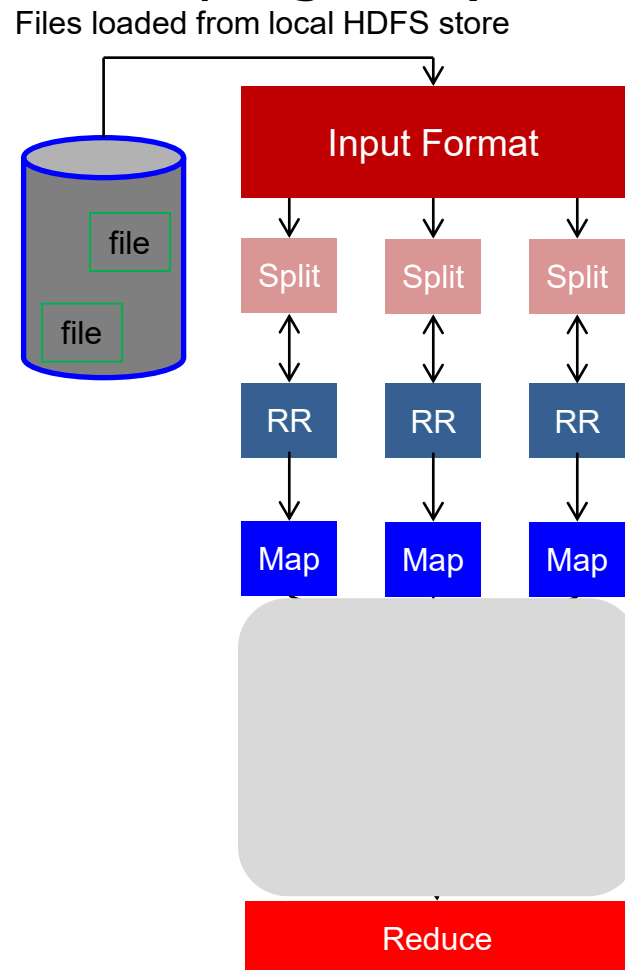
Record Reader

- The input split defines a slice of work but does not describe how to access it.
- The Record Reader class loads data from its source and converts it to (K, V) pairs suitable for reading by Mappers.



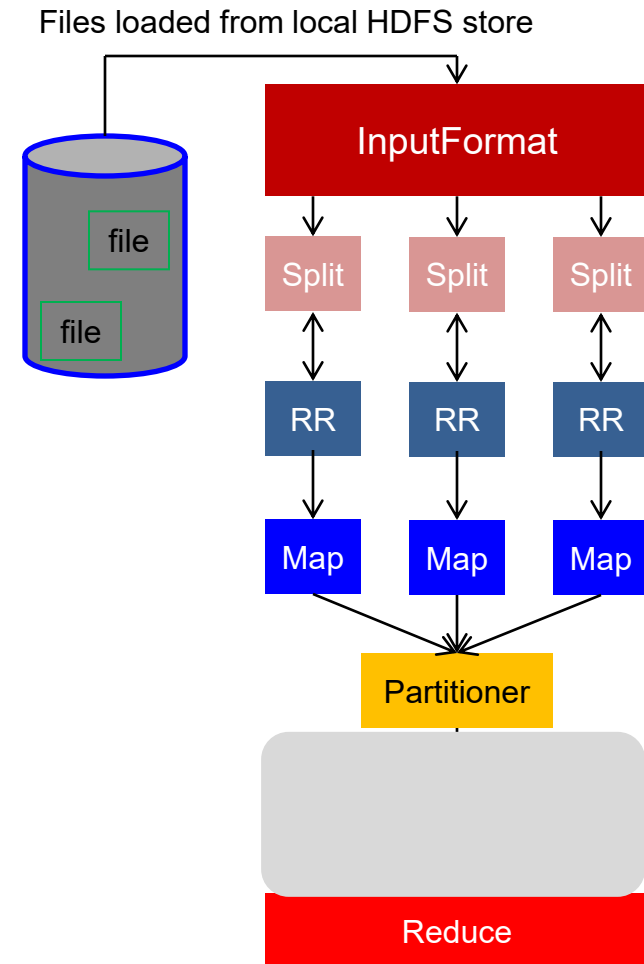
Mappers

- The Mapper performs the user-defined work within the first phase of the MapReduce program (i.e., the Map phase).



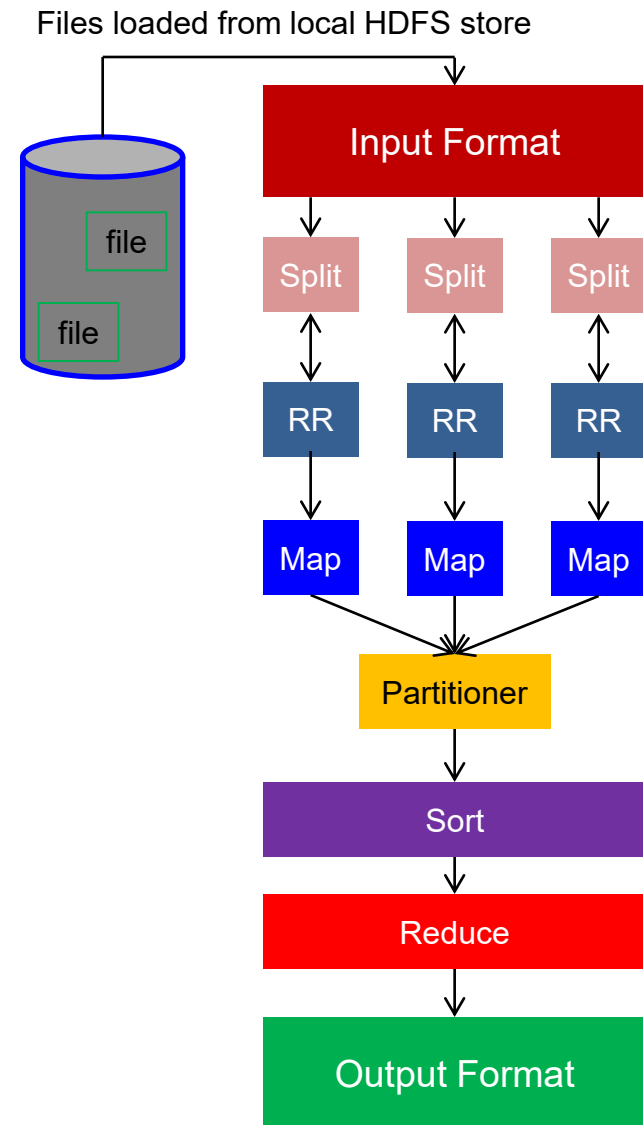
Partitioner

- Each mapper may emit (K, V) pairs to any Reduce task.
- Therefore, the map nodes must all agree on where to send different pieces of **intermediate** output.
- The Partitioner class determines which Reduce task (or partition) a given (K,V) pair will be assigned to.



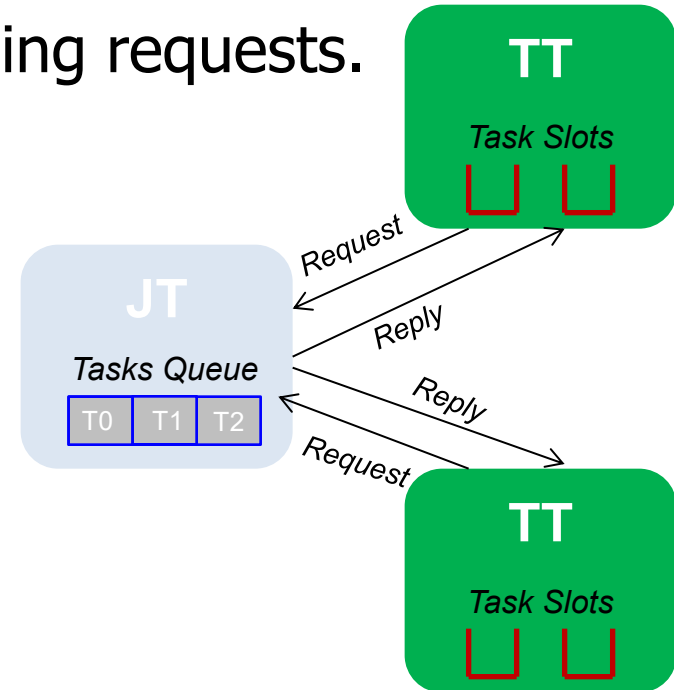
Output Format

- Each Reduce task is responsible for reducing the values associated with (several) intermediate keys.
- Intermediate keys of a single partition are automatically **merged & sorted**.
- Reduce tasks apply the user-defined reduce function on merged and sorted partitions and output the final results.
- The Output Format class defines the way (K,V) pairs produced by Reducers are written to output files.



Task Scheduling in MapReduce

- MapReduce adopts a master-slave architecture.
- The master node is referred to as JobTracker (JT).
- Each slave node is referred to as TaskTracker (TT).
- MapReduce adopts a pull-based scheduling strategy (rather than a push-based one).
- I.e., JT does not push Map and Reduce tasks to TTs but rather TTs pull them by making pertaining requests.



Task Scheduling in MapReduce

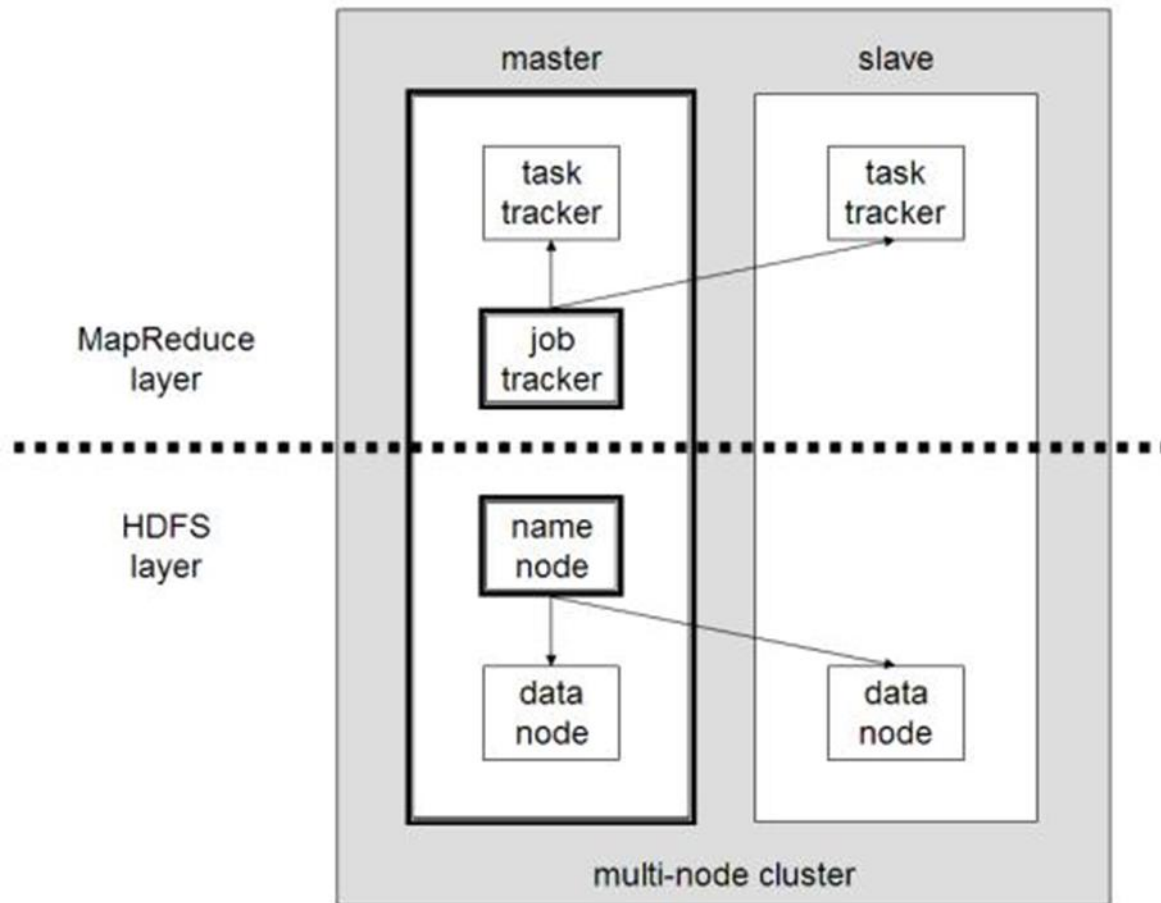
■ Map/Reduce Master “**Jobtracker**”

- **Accepts** MR jobs submitted by users
- **Assigns** Map and Reduce tasks to Tasktrackers
- **Monitors** task and tasktracker status, reexecutes tasks upon failure

■ Map/Reduce Slaves “**Tasktrackers**”

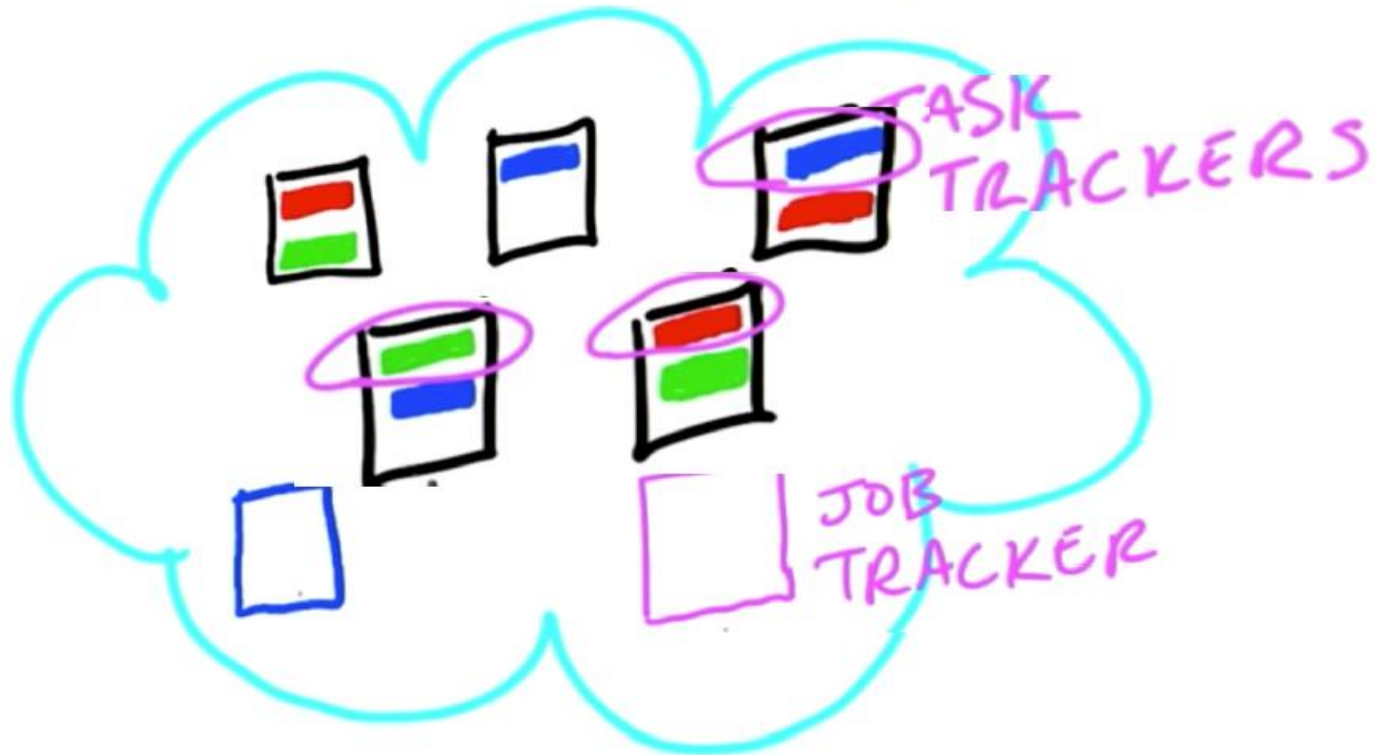
- **Run** Map and Reduce tasks upon instruction from the Jobtracker
- **Manage** storage and transmission of intermediate output.

Task Scheduling in MapReduce



Task Scheduling in MapReduce

Daemons of Map Reduce

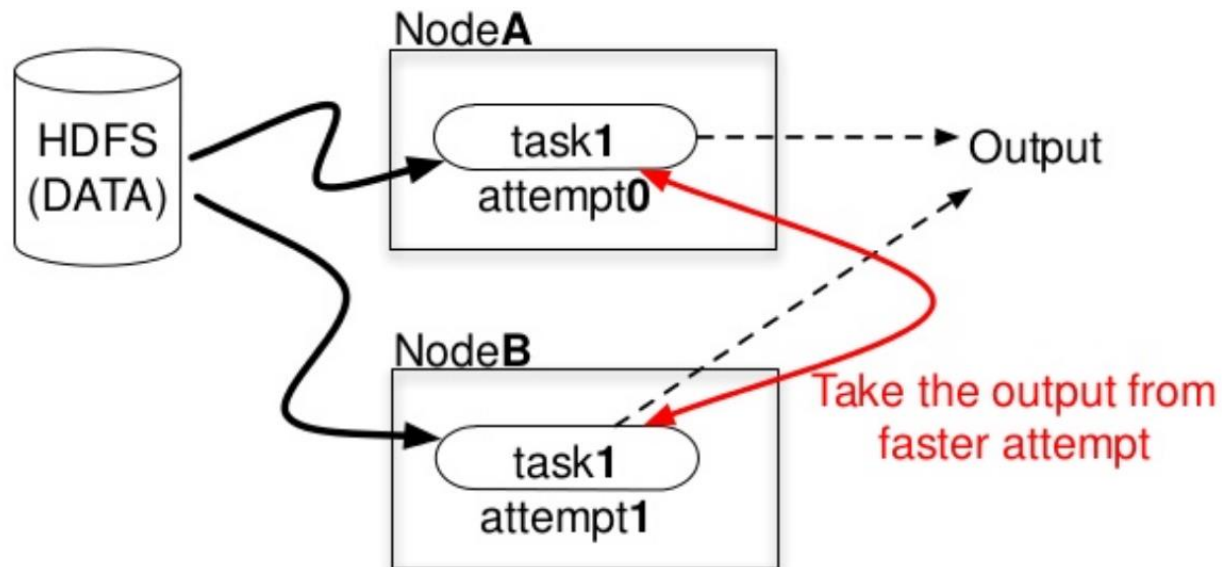


Fault Tolerance in Hadoop: Node Failures

- MapReduce can guide jobs toward a successful completion even when jobs are run on large clusters where probability of failures increases.
- Hadoop MapReduce achieves fault-tolerance through restarting tasks.
- If a TT fails to communicate with JT for a period of time (by default, 1 minute), JT will assume that TT in question has crashed.
- If the job is still in the Map phase, JT asks another TT to re-execute all Map tasks that previously ran at the failed TT.
- If the job is in the Reduce phase, JT asks another TT to re-execute all Reduce tasks that were in-progress on the failed TT.

Fault Tolerance in Hadoop: Speculative Execution

- A MapReduce job is dominated by the slowest task.
- MapReduce attempts to locate slow tasks (or stragglers) and run replicated (or speculative) tasks that will optimistically commit before the corresponding stragglers.



A simple example 7 – Input Data

2012-01-01 12:01 San Jose Music 12.99 Amex

- Each mapper processes a portion of the input data.
 - Each one will be given a line at a time. The lines look like this.
 - The mapper needs to take that line, and extract the information it needs.
 - we're dealing with text, it's pretty free-form. So we'd use something like a regular expression.
 - But in this case, it's regular, it's tab limited.
 - So we can split the line on tab and extract the values
- Calculate the **total sales per store**.

A simple example 7 – Mapper

```
def mapper():  
    for line in sys.stdin:  
        data = line.strip().split("\t")
```


A simple example 7 – Mapper

- Suppose that the data contains noises, like:

```
2012-01-01 12:01 San Jose Music 12.99 Amex
2012-01-02 There was an error trying to connect to the database. Please try again.
```

A simple example 7 – Mapper

```
def mapper():  
    for line in sys.stdin:  
        data = line.strip().split("\t")  
  
        if len(data) == 6:  
            date, time, store, item, cost, payment = data  
            print "{0}\t{1}".format(store, cost)
```

A simple example 7 – Shuffle and sort

- the data comes in streamed as lines of text
- Each line contains a store name and the cost
- The store names are sorted, which we're guaranteed because of the shuffle and sort.
- So we know that all lines for, say, Miami, are together:

Miami	12.34
Miami	99.07
Miami	3.14
NYC	99.77
NYC	88.99

A simple example 7 – Reducer

Miami	12.34
Miami	99.07
Miami	3.14
NYC	99.77
NYC	88.99

```
def reducer():  
    salesTotal = 0  
    oldKey = None  
  
    for line in sys.stdin:  
        data = line.strip().split("\t")  
  
        if len(data) != 2:  
            continue  
  
        thisKey, thisSale = data  
  
        if oldKey and oldKey != thisKey:  
            print "{0}\t{1}".format(oldKey, salesTotal)  
  
            salesTotal = 0  
  
        oldKey = thisKey  
        salesTotal += float(thisSale)  
  
    if oldKey != None:  
        print "{0}\t{1}".format(oldKey, salesTotal)
```



A simple example 7 – Job Running

- I will have to locate my code on the machine
- To submit a map reduce job to the cluster, I would use:
 - Hadoop jar, a path to a jar, then I specify the mapper, I specify the reducer, I need to say -file, for both the mapper and the reducer code. I specify the input directory in HDFS and I specify the output directory to which the reducers will write their output data. And we're calling that joboutput.

```
[training@localhost code]$ hadoop jar /usr/lib/hadoop-0.20-mapreduce/contrib/streaming/hadoop-streaming-2.6.0-mr1-cdh4.1.1.jar -mapper mapper.py -reducer reducer.py -file mapper.py -file reducer.py -input myinput -output joboutput
```

A simple example 7 – Job Results

- To check the output of the we will use the command:
 - `Hadoop fs -ls`: it shows we have got a job output directory
 - `Hadoop fs -ls joboutput`: shows that it contains three files:
 - `_SUCCESS`: means that the job has successfully completed
 - `_logs`: contains some log information about what happened during the job's run
 - `part-00000`: is the output from the one reducer that we had for this job.

A simple example 7 – Job Results

- To take a look at the file `part-00000`, we pipe the file to `less` in our local machine. Here is the command:
 - `hadoop fs -cat joboutput/part-00000 | less`
- To retrieve data from HFDS and put it onto our local disk, we can use this command:
 - `Hadoop fs -get joboutput/part-00000 mylocalfile.txt`
- To open the file now located on my local disk, I can use this command:
 - `less mylocalfile.txt`

A simple example 7 – Web GUI

- To see what's going on on the cluster, we can check the hadoop job tracker web user interface:
 - `Localhost:50030/jobtracker.jsp`

0 Hadoop Map/Reduce Administration

State: RUNNING
Started: Wed Sep 11 16:31:06 EDT 2013
Version: 2.0.0-mr1-cdh4.1.1, Unknown
Compiled: Tue Oct 16 11:50:49 PDT 2012 by jenkins from Unknown
Identifier: 201309111631

Cluster Summary (Heap Size is 15.31 MB/193.38 MB)

Running Map Tasks	Running Reduce Tasks	Total Submissions	Nodes	Occupied Map Slots	Occupied Reduce Slots	Reserved Map Slots	Reserved Reduce Slots	Map Task Capacity	Reduce Task Capacity	Avg. Tasks/Node	Blacklisted Nodes	Excluded Nodes
2	0	4	1	2	0	0	0	2	2	4.00	0	0

Scheduling Information

Queue Name	State	Scheduling Information
default	running	N/A

Filter (Jobid, Priority, User, Name)

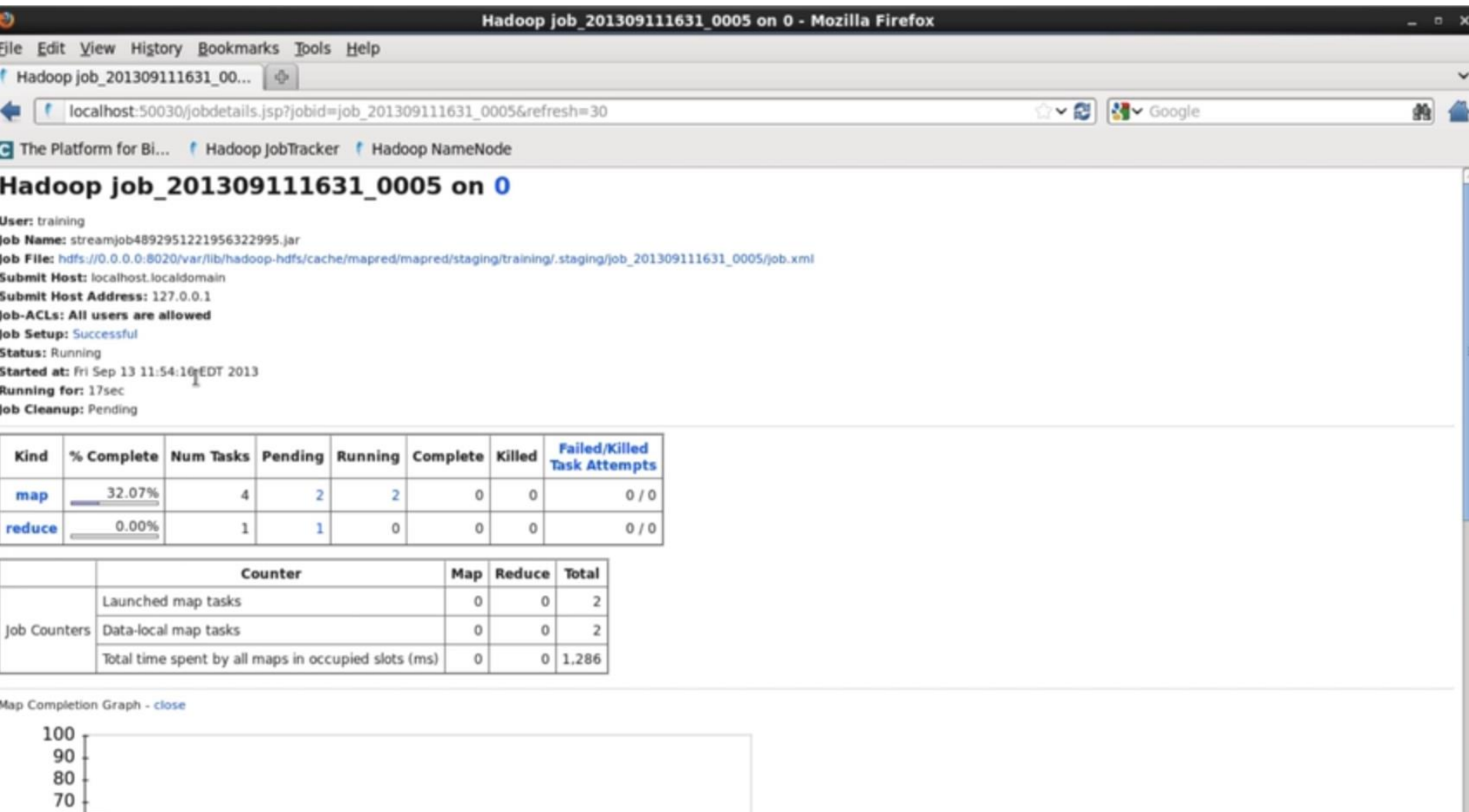
Example: 'user:smith 3200' will filter by 'smith' only in the user field and '3200' in all fields

Running Jobs

Jobid	Priority	User	Name	Map % Complete	Map Total	Maps Completed	Reduce % Complete	Reduce Total	Reduces Completed	Job Scheduling Information	Diagnostic Info
job_201309111631_0005	NORMAL	training	streamjob4892951221956322995.jar	16.11% 	4	0	0.00% 	1	0	NA	NA

Completed Jobs

A simple example 7 – Job details



A simple example 7 – Job details

Hadoop Task Details - Mozilla Firefox

File Edit View History Bookmarks Tools Help

Hadoop Task Details

localhost:50030/taskdetails.jsp?tipid=task_201309111631_0005_m_000000

The Platform for Bi... Hadoop JobTracker Hadoop NameNode

Job job_201309111631_0005

All Task Attempts

Task Attempts	Machine	Status	Progress	Start Time	Finish Time	Errors	Task Logs	Counters	Actions
attempt_201309111631_0005_m_000000_0	/default-rack/localhost.localdomain	SUCCEEDED	100.00% 	13-Sep-2013 11:54:17	13-Sep-2013 11:54:42 (24sec)		Last 4KB Last 8KB All	22	

Input Split Locations

/default-rack/localhost.localdomain

[Go back to the job](#)
[Go back to JobTracker](#)

Hadoop, 2013.

A simple example 7 – Job task list

Hadoop map task list for job_201309111631_0005 on 0 - Mozilla Firefox

File Edit View History Bookmarks Tools Help

Hadoop map task list for job_2...

localhost:50030/jobtasks.jsp?jobid=job_201309111631_0005&type=map&pagenum=1

The Platform for Bi... Hadoop JobTracker Hadoop NameNode

Hadoop map task list for job_201309111631_0005 on 0

All Tasks

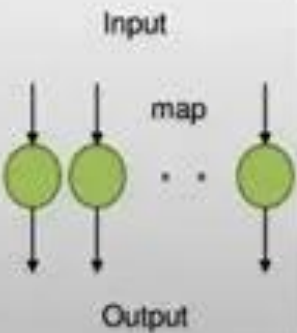
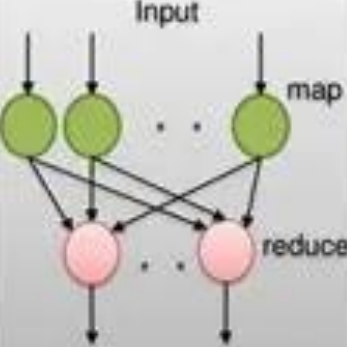
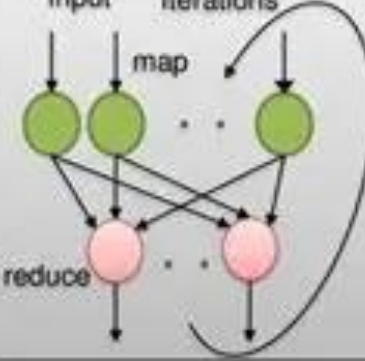
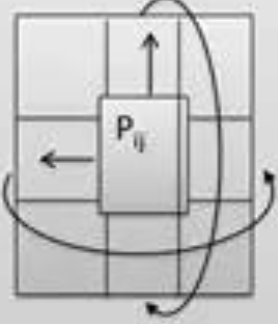
Task	Complete	Status	Start Time	Finish Time	Errors	Counters
task_201309111631_0005_m_000000	100.00% 	Records R/W=728768/724567	13-Sep-2013 11:54:17	13-Sep-2013 11:54:42 (24sec)		22
task_201309111631_0005_m_000001	100.00% 	Records R/W=728783/726768	13-Sep-2013 11:54:17	13-Sep-2013 11:54:42 (24sec)		22
task_201309111631_0005_m_000002	0.00% 	initializing	13-Sep-2013 11:54:42			0
task_201309111631_0005_m_000003	0.00% 	initializing	13-Sep-2013 11:54:42			0

[Go back to JobTracker](#)

Hadoop, 2013.

Advanced MapReduce

4 Forms of MapReduce

(a) Map Only	(b) Classic MapReduce	(c) Iterative MapReduce	(d) Loosely Synchronous
 Input map Output	 Input map reduce Output	 Input Iterations map reduce	 P_{ij}
BLAST Analysis Parametric sweep Pleasingly Parallel	High Energy Physics (HEP) Histograms Distributed search	Expectation maximization Clustering e.g. Kmeans Linear Algebra, Page Rank	Classic MPI PDE Solvers and particle dynamics
← Domain of MapReduce and Iterative Extensions → Science Clouds			MPI Exascale

Other Hadoop ecosystem tools

Apache Spark

- Apache Spark emerges as another open-source framework for big data processing, extending beyond the MapReduce model.
- It introduces in-memory data processing, potentially faster than the conventional disk-based MapReduce processing.
- Spark supports multiple programming languages, including Scala, Java, Python, and R, and comes equipped with libraries tailored for machine learning, graph processing, and various other applications.

Apache Flink

- Apache Flink stands out as a dual-purpose framework, supporting both stream processing and batch processing.
- It facilitates MapReduce-style operations while emphasizing low-latency and high-throughput processing.
- Notably, Apache Flink provides a unified API for seamless integration of both batch and stream processing.



- Pig is the framework for executing MapReduce on HDFS data using its own scripting language.
- Why would you want to use Pig rather than MapReduce?
- Native MapReduce applications written in Java are effective and powerful tools, but the time to develop and test the applications is time-consuming and complex.
- Pig solves this problem by offering a simpler development and testing process that takes advantage of the power of MapReduce, without the need to build large Java applications.
- Whereas Java programs may require 50-100 lines, Pig scripts often have ten lines of code or less.

Sqoop



- Used for Data Integration.
- It's likely that some of your data may originate in a relational database management system (RDBMS) that is usually accessed normally by SQL.
- Sqoop(meaning SQL to Hadoop) is designed to transfer data between relational databases and Hadoop clusters.
- You'll start your import to Hadoop with a database table that is read into Hadoop as a text file. You can also export an HDFS file into an RDBMS.

Flume

- Used for Data Integration.
- Flume is a reliable distributed system for collecting, aggregating, and moving large amounts of log data from multiple sources into HDFS.

Cassandra

- Used for Data Storage.
- Oftentimes you may need to simply organize some of your big data for easy retrieval.
- One common way to do this is to use a key-value datastore.
- Cassandra is a distributed key-value database designed with simplicity and scalability in mind.
- Your data is organized by a unique “key,” and values are associated with that key.
- Cassandra is an all-inclusive system, which means it does not require a Hadoop environment or any other big data tools.

- Used for Data Storage.
- HBase is a NoSQL database system included in the standard Hadoop distributions that works with HDFS for data storage and access. It is based on Google's BigTable.
- It is a key-value store, logically. This means that rows are defined by a key, and have associated with them a number of bins (or columns) where the associated values are stored.
- Physically, groups of similar columns are stored together in column families. Most often, HBase is accessed via Java code, but APIs exist for using HBase with Pig.
- HBase is often used for applications that may require sparse rows. That is, each row may use only a few of the defined columns.



- Used for Data Storage.
- The goal of Hive is to allow SQL access to data in the HDFS.
- Hive defines a simple SQL-like query language, called HQL, that enables users familiar with SQL to query the data.
- Queries written in HQL are converted into MapReduce code by Hive and executed by Hadoop.

MongoDB

- Used for Data Storage.
- MongoDB is a document-oriented database, the document being a JSON object.
- In relational databases, you have tables and rows. In MongoDB, the equivalent of a row is a JSON document, and the analog to a table is a collection, a set of JSON documents.
- At the start of 2015, it is one of the most popular NoSQL databases.
- Unlike some other database systems, MongoDB supports secondary indexes—meaning it is possible to quickly search on other than the primary key that uniquely identifies each document in the Mongo database.



- You might want to do some analytics, or data science, or machine learning.
- Much of this can be done in some of the tools that come with the standard Apache distribution, such as Pig, MapReduce, or Hive.
- But more sophisticated uses will involve algorithms that you will not want to code yourself.
- So you turn to Mahout.
- Mahout is a collection of scalable machine-learning algorithms that run on Hadoop.

Apache Chukwa



- Log analysis framework built on HDFS & Map/Reduce and HBase
- Data collection system to monitor large distributed systems
- Toolkit for displaying & analyzing collected data



- Data serialization system
 - Serialization: "...the process of converting a data structure or object state into a format that can be stored" in a file or transmitted over a network link.
 - Serializing an object is also referred to as "marshaling" an object
- Supports JSON as the data interchange format
 - JavaScript Object Notation, a self-describing data format
 - Experimental support for Avro IDL, an alternate Interface Description Lang.
- Includes Remote Procedure Call (RPC) in the framework
 - Communication between Hadoop nodes and from clients to services.

Apache Whirr



- Libraries for running cloud services
 - Cloud-neutral, avoids provider idiosyncrasies
- Command line tool for deploying large clusters
- Common service API
- Smart defaults for services to enable quickly getting a properly configured system running but still specify the desired attributes

Apache Zookeeper



- Goal: Provide highly reliable distributed coordination services via a set of primitives for distributed apps to build on
 - “Because Coordinating Distributed Systems is a Zoo” (!)
 - Common distributed services notoriously difficult to develop & maintain on an application by application basis
- Centralized distributed synchronization;
- group services;
- naming;
- configuration information

HCatalog; MRUnit; Oozie

■ HCatalog

- Table and storage management service for data created using Apache Hadoop
 - Table abstraction with regard to where or how the data is stored
 - To provide interoperability across data processing tools including Pig; Map/Reduce; Streaming; Hive

■ MRUnit

- Library to support unit testing of Hadoop Map/Reduce jobs

■ Oozie

- Workflow system to manage Hadoop jobs by time or data arrival

Thank you